Computer Networks (CN) - తెలుగు గైడ్ (SSE Fundamentals)

> ఈ document చదివిన తర్వాత Computer Networks మళ్ళీ జీవితంలో మర్చిపోలేవు. Networking theory అస్సలు తెలియని వ్యక్తిని — ZERO నుండి — Senior Software Engineer (SSE) interview crack చేసే స్థాయికి తీసుకెళ్లడమే లక్ష్యం. ప్రతి concept కి ఒక vivid real-life analogy (postal system, phone call, రోడ్లు), ఎందుకు/ఎప్పుడు, ASCII diagram, comparison table, worked example (subnetting math), మరియు interview దృష్టి — అన్నీ ఉంటాయి. "ఒకసారి చదివితే జీవితంలో మర్చిపోకూడదు."
>
> **నీ నేపథ్యం (ఈ guide ఎవరి కోసం):** నువ్వు MERN developer వి — JavaScript, React, Node, `fetch()` , HTTP, REST APIs రోజూ వాడతావు. కానీ CS background లేదు, networking theory ZERO. అంటే నువ్వు ఇప్పటికే **networks ని వాడుతున్నావు** (ప్రతి `fetch()` ఒక network call), కానీ **లోపల ఏం జరుగుతుందో** తెలియదు. ఈ guide ఆ gap ని పూరిస్తుంది — నీకు తెలిసిన web knowledge (fetch, HTTP, WebSockets, cookies) ని పునాదిగా వాడి, కింద ఉన్న theory అంతా connect చేస్తుంది.
>
> **లక్ష్యం:** "నేను ఒక `fetch("https://api.com/users")` రాస్తే, browser నుండి server దాకా ఏం జరుగుతుంది?" అనే ప్రశ్నకి — DNS, TCP handshake, TLS, IP packets, routers, ports — ప్రతి layer లో ఏం జరుగుతుందో పూర్తిగా, గట్టిగా చెప్పగలగాలి. అదే SSE interview అడిగేది.
>
> ఇది CS-fundamentals set లో భాగం — `OOPS_Telugu.md` , `GO_Telugu.md` , `HLD_Go_Telugu.md` , `SystemDesign_Go_Telugu.md` , మరియు `DSA_00..10` docs కి కొనసాగింపు. Best-teacher style — intuition first, formalism తర్వాత.

విషయ సూచిక (Table of Contents)

Part 1 — Basics (పునాదులు)

1. Network అంటే ఏమిటి, ఎందుకు (LAN/WAN/MAN, topologies, ఎందుకు layered architecture)
2. OSI Model — 7 layers deep (ప్రతి layer పని, mnemonic, ఎలా గుర్తుంచుకోవాలి)
3. TCP/IP Model (OSI తో పోలిక, encapsulation/decapsulation with diagram)

Part 2 — Layers (ప్రతి layer లోతుగా)

4. Physical & Data Link Layer (signals, MAC address, switches, ARP, CRC error detection, Ethernet, collision/CSMA-CD)
5. Network Layer (IPv4/IPv6, IP classes, subnetting + CIDR with worked math, routing, NAT, ICMP/ping, DHCP)
6. Transport Layer (TCP vs UDP deep, ports, 3-way handshake + 4-way termination, sequence/ack, flow control/sliding window, congestion control)
7. Application Layer (overview: HTTP, DNS, SMTP, FTP, WebSockets)

Part 3 — Web & Practical (నీ MERN ప్రపంచం)

8. HTTP deep (request/response structure, methods, status codes, headers, cookies/sessions, HTTP/1.1 vs 2 vs 3, keep-alive, REST)
9. HTTPS & TLS (ఎందుకు, TLS handshake steps, certificates/CA/PKI, symmetric+asymmetric combo)
10. DNS deep (hierarchy: root/TLD/authoritative, resolution steps, record types A/AAAA/CNAME/MX, caching/TTL)
11. "URL enter చేస్తే ఏం జరుగుతుంది" — full end-to-end journey (DNS → TCP → TLS → HTTP → render). Classic interview question.

Part 4 — Reference (security & interview)

12. Network Security & Tools basics (firewall, VPN, proxy, attacks DDoS/MITM, commands ping/traceroute/netstat/curl)
13. Interview Q&A + Memory Tips table + Common Mistakes

Part 1 — Basics (పునాదులు)

> Networking అంటే ఒక్క మాటలో: **"రెండు computers ఎలా ఒకదానితో ఒకటి మాట్లాడుకుంటాయి?"** ఈ Part లో ముందు network అంటే ఏమిటో, ఎందుకు అవసరమో అర్థం చేసుకుంటాం. తర్వాత ఆ మాట్లాడే process ని engineers ఎలా **layers గా విడగొట్టారో** (OSI, TCP/IP models) చూస్తాం. ఈ layered ఆలోచన పక్కాగా ఉంటే, మిగతా CN అంతా సులభం — ఎందుకంటే ప్రతి protocol (HTTP, TCP, IP, Ethernet) ఏదో ఒక layer లో ఉంటుంది.

---

1. Network అంటే ఏమిటి, ఎందుకు

వివరణ

Computer Network = రెండు లేదా అంతకంటే ఎక్కువ computers ఒకదానితో ఒకటి data పంచుకోవడానికి connect అయ్యి ఉండటం. అంతే. అంతకంటే complex ఏమీ లేదు.

నీకు ఇది ఇప్పటికే తెలుసు — నువ్వు గుర్తించట్లేదు అంతే. React app లో నువ్వు రాసే ఇది చూడు:

```
fetch("https://api.github.com/users/torvalds")
  .then(res => res.json())
  .then(data => console.log(data));
```

ఈ ఒక్క line లో — **నీ computer (client)** ఒక **వేరే computer (GitHub server)** తో మాట్లాడుతోంది. నీ లాప్‌టాప్ Chennai లో ఉంది, GitHub server అమెరికాలో ఉంది. మధ్యలో సముద్రాలు, వందల routers, cables — అయినా 200ms లో data వచ్చేస్తుంది. **ఇదే network.** ఈ మొత్తం guide చేసేది — ఆ 200ms లో ఏం జరిగిందో విడగొట్టి చూపించడం.

కొన్ని basic పదాలు (vocabulary) — ఇవి guide అంతా వస్తాయి:

పదం	అర్థం (MERN connection తో)
Node / Host	Network లో ఉన్న ఏ device అయినా — నీ laptop, phone, server, router. నీ Node.js server ఒక host.
Link	రెండు nodes మధ్య ఉన్న connection — cable (Ethernet), లేదా wireless (WiFi).
Client	Data అడిగే device — నీ browser, నీ React app.
Server	Data ఇచ్చే device — నీ Express/Node backend, GitHub API.
Protocol	రెండు devices ఎలా మాట్లాడాలో ముందే ఒప్పుకున్న **rules set**. HTTP, TCP, IP — అన్నీ protocols.
Packet	Data ని పంపేటప్పుడు చిన్న చిన్న ముక్కలుగా విడగొడతారు — ఆ ఒక్కో ముక్క ఒక packet.
Bandwidth	ఒక link ద్వారా second కి ఎంత data పంపగలవు (Mbps, Gbps). రోడ్డు వెడల్పు లాంటిది.
Latency	Data ఒక చోటి నుండి ఇంకో చోటికి చేరడానికి పట్టే time (ms). రోడ్డు పొడవు / traffic లాంటిది.

Protocol అంటే ఎందుకు అంత ముఖ్యం? ఊహించు — నువ్వు ఒకరికి phone చేశావు. అవతలి వ్యక్తి "Hello" అంటాడు, నువ్వు మాట్లాడతావు, ఇద్దరూ ఒకేసారి మాట్లాడరు, చివర్లో "bye" అని పెట్టేస్తారు. ఇవన్నీ **మనం ఒప్పుకున్న rules** — protocol.

Computers కి కూడా ఇలాంటి rules కావాలి, లేకపోతే ఒకరు పంపిన 0s and 1s అవతలివారికి అర్థం కావు. **Protocol = common language + common rules.**

Real-life Scenario

> **Network = రోడ్ల వ్యవస్థ (road system) + తపాలా (postal system).** ఈ analogy మొత్తం guide అంతా వాడతాం, కాబట్టి గట్టిగా పట్టుకో:
>
> నువ్వు Hyderabad లో ఉన్నావు, నీ స్నేహితుడు Delhi లో ఉన్నాడు. నువ్వు అతనికి ఒక పెద్ద పుస్తకం పంపాలి.
>
> - నువ్వు పుస్తకాన్ని ఒకేసారి పంపవు — **పేజీలుగా విడగొట్టి** (packets), ఒక్కో పేజీని ఒక్కో cover లో పెట్టి పంపుతావు.
> - ప్రతి cover మీద **గమ్యస్థానం address** (destination IP), **నీ address** (source IP), **పేజీ నంబరు** (sequence number) రాస్తావు.
> - Covers వేర్వేరు దారుల్లో (routes) వెళ్ళి, వేర్వేరు time లకి Delhi చేరతాయి — కానీ పేజీ నంబర్లు ఉన్నాయి కాబట్టి నీ స్నేహితుడు వాటిని **మళ్ళీ order లో పేర్చుకోగలడు** (reassembly).
> - ఏదైనా cover పోతే, నీ స్నేహితుడు "ఫలానా పేజీ రాలేదు, మళ్ళీ పంపు" అని అడుగుతాడు (retransmission — TCP చేసేది ఇదే).
>
> **ఇదే packet-switched network యొక్క ఆత్మ.** Internet పని చేసేది సరిగ్గా ఇలాగే. ఈ guide లో ప్రతి layer ఈ postal system లో ఒక్కో పని చేస్తుంది — ఎవరు cover మీద address రాస్తారు (IP), ఎవరు పేజీ నంబరు వేస్తారు (TCP), ఎవరు నిజంగా రోడ్డు మీద తీసుకెళ్తారు (physical layer) — అన్నీ చూస్తాం.

ఎందుకు networks అవసరం? (Why)

Networks లేకపోతే ప్రతి computer ఒక ఒంటరి ద్వీపం (isolated island). Network తో ఏమి సాధ్యమవుతుందంటే:

1. **Resource sharing** — ఒక printer ని 50 మంది వాడొచ్చు. ఒక database ని అందరూ access చేయొచ్చు.
2. **Communication** — email, WhatsApp, video calls, నీ React app ↔ Node backend.
3. **Data access from anywhere** — నీ data cloud (AWS/GCP) లో ఉంటే ప్రపంచంలో ఎక్కడి నుండైనా access చేయొచ్చు.
4. **Distributed computing** — ఒక పెద్ద పని ని 100 servers పంచుకుని చేయడం (నీ HLD docs లో చదివిన scaling).

MERN developer కి direct connection: **నీ frontend (React, browser లో) మరియు backend (Node, server లో) రెండు వేర్వేరు computers.** వాటి మధ్య network లేకపోతే నీ app పని చేయదు. ప్రతి API call, ప్రతి WebSocket message, ప్రతి image load — network మీద ఆధారపడి ఉంటుంది.

Network రకాలు — పరిమాణం (geographic size) ఆధారంగా

Networks ని అవి ఎంత దూరం విస్తరించి ఉన్నాయో దాన్నిబట్టి classify చేస్తారు:

```
PAN  →  LAN  →  MAN  →  WAN
(చిన్నది)              (పెద్దది)
1 మనిషి   1 building   1 నగరం      దేశాలు/ప్రపంచం
```

రకం	పూర్తి పేరు	పరిధి (range)	ఉదాహరణ	ఎవరు own చేస్తారు
PAN	Personal Area Network	~10 meters	నీ phone ↔ Bluetooth earbuds, smartwatch	నువ్వే
LAN	Local Area Network	1 building/campus	నీ ఇంటి WiFi, office network	ఒక organization
MAN	Metropolitan Area Network	1 నగరం	ఒక city cable TV network, university campuses కలిపి	ISP / city
WAN	Wide Area Network	దేశాలు, ఖండాలు	**Internet** (అతిపెద్ద WAN), banks అన్ని branches కలిపే network	ISPs, telecom companies

గుర్తుంచుకో: Internet = అతిపెద్ద WAN. ఇది కోట్ల LAN లని కలిపే "network of networks". నీ ఇంటి WiFi (LAN) → నీ ISP (MAN/WAN) → backbone cables → GitHub యొక్క data center LAN. ప్రతి `fetch()` ఈ chain మీదుగా ప్రయాణిస్తుంది.

Network Topology — devices ఎలా arrange అయ్యి ఉంటాయి

Topology = network లో devices ఒకదానితో ఒకటి ఏ ఆకారంలో (physical/logical layout) connect అయ్యి ఉన్నాయి. ఇది ఎందుకు ముఖ్యం? ఎందుకంటే layout మారితే — cost, speed, ఒక cable తెగితే ఏమవుతుంది (fault tolerance) — అన్నీ మారతాయి.

```
BUS Topology (అందరూ ఒకే cable మీద)
   A     B     C     D
   |     |     |     |
===+=====+=====+=====+===   ← ఒకే backbone cable
   ఒక cable తెగితే మొత్తం network down.

STAR Topology (అందరూ ఒక central hub/switch కి)
        A
        |
   B---[HUB]---C          ← ప్రతి device switch కి
        |
        D
   ఒక cable తెగితే ఆ ఒక్క device మాత్రమే down. (నీ ఇంటి WiFi ఇదే)

RING Topology (వలయంలా, ఒకరి తర్వాత ఒకరు)
    A --- B
    |     |
    D --- C
   Data ఒక దిశలో తిరుగుతుంది. ఒకటి తెగితే ring విరిగిపోతుంది.

MESH Topology (అందరూ అందరితో)
    A ----- B
    | \   / |
    |  \ /  |
    |  / \  |
    | /   \ |
```

```
  D ----- C
చాలా cables, ఖరీదు ఎక్కువ, కానీ ఒకటి తెగినా వేరే దారి. (Internet backbone)
```

Topology	ప్రయోజనం (Pros)	నష్టం (Cons)	ఎక్కడ వాడతారు
Bus	తక్కువ cable, cheap, simple	ఒక cable తెగితే అంతా down; collisions ఎక్కువ	పాత చిన్న networks (ఇప్పుడు వాడరు)
Star	ఒక device fail అయినా మిగతావి safe; manage చేయడం easy	Central switch fail అయితే అంతా down	**ఇళ్ళు, offices** (అత్యంత common)
Ring	Collision తక్కువ, predictable	ఒక node fail అయితే ring విరుగుతుంది	పాత token-ring networks
Mesh	అత్యధిక fault tolerance, ఒకటి తెగినా backup దారి	చాలా ఖరీదు, cables ఎక్కువ	**Internet backbone**, critical systems

Circuit Switching vs Packet Switching (classic interview)

Data ని network గుండా పంపడానికి రెండు fundamental approaches:

అంశం	Circuit Switching	Packet Switching
ఎలా	ముందు ఒక dedicated path reserve, తర్వాత data	Data ని packets గా విడగొట్టి, ఒక్కోటి స్వతంత్రంగా
Path	Fixed (మొత్తం call కి same)	ప్రతి packet వేరే route వెళ్ళొచ్చు
ఉదాహరణ	పాత landline phone call	**Internet** (IP)
Resource	Reserved (వాడకపోయినా blocked)	Shared (efficient)
Reliability	Path down → call drop	Path down → వేరే route
Setup delay	ఉంది (path reserve)	లేదు (వెంటనే పంపు)

> **Analogy — landline vs postal. Circuit switching = పాత landline phone.** నువ్వు call చేస్తే, నీకు-అవతలివారికి మధ్య ఒక dedicated wire path reserve అవుతుంది — మాట్లాడకపోయినా అది నీకే (waste). **Packet switching = postal system.** నీ letter ని ముక్కలుగా (packets) పంపి, ఒక్కోటి వేరే route లో వెళ్ళి, అవతల కలుపుతారు. రోడ్లు అందరూ share చేస్తారు (efficient). **Internet packet switching వాడుతుంది** — అందుకే billions of users ఒకే infrastructure share చేయగలరు.

ఎందుకు internet packet switching? Resource sharing (efficient), fault tolerance (ఒక route down → వేరేది), no setup delay, cost-effective. Circuit switching యొక్క guaranteed bandwidth మంచిదే కానీ waste ఎక్కువ.

ఎందుకు Layered Architecture? (అత్యంత ముఖ్యమైన idea)

ఇది CN మొత్తానికి పునాది. **రెండు computers మాట్లాడుకోవడం చాలా complex పని** — signals పంపాలి, errors handle చేయాలి, address కనుక్కోవాలి, lost data మళ్ళీ పంపాలి, data ని application అర్థం చేసుకునేలా format చేయాలి.

ఇదంతా ఒకే పెద్ద program గా రాస్తే — అది maintain చేయడం, debug చేయడం అసాధ్యం.

పరిష్కారం: పనిని layers గా విడగొట్టడం. ప్రతి layer ఒక specific పని మాత్రమే చేస్తుంది, పైన/కింద layer కి clean interface ఇస్తుంది.

> **Real-life analogy — courier company (Amazon delivery).** నువ్వు Amazon లో వస్తువు order చేశావు. ఇది layers గా జరుగుతుంది:
>
> - **Layer 3 (నువ్వు):** "ఈ వస్తువు కావాలి" అని decide చేస్తావు. Truck ఎలా నడుస్తుందో నీకు అనవసరం.
> - **Layer 2 (Amazon warehouse):** వస్తువుని box లో packing చేసి, address label అంటిస్తారు. ఏ truck అనేది వాళ్ళకి అనవసరం.
> - **Layer 1 (delivery truck driver):** Box ని రోడ్డు మీద తీసుకెళ్తాడు. లోపల ఏముందో అతనికి అనవసరం.
>
> **ప్రతి layer తన పని మాత్రమే చేస్తుంది, పక్క layer వివరాలు తెలుసుకోవాల్సిన అవసరం లేదు (abstraction).** Truck driver మారినా (bike delivery కి మారినా) నీకు, warehouse కి తేడా లేదు. ఇదే networking layers ఇచ్చే గొప్ప ప్రయోజనం.

Layering ఇచ్చే 4 ప్రయోజనాలు (interview లో అడుగుతారు):

1. **Modularity** — ప్రతి layer ని విడిగా design/build చేయొచ్చు. Transport layer team, network layer team వేరు వేరుగా పని చేయొచ్చు.
2. **Abstraction** — పై layer కి కింద layer ఎలా పని చేస్తుందో తెలియాల్సిన అవసరం లేదు. HTTP కి TCP ఎలా data పంపుతుందో అనవసరం.
3. **Interoperability** — ఒక layer ని మార్చినా (WiFi → Ethernet, physical layer మారింది) పైన ఉన్న layers (HTTP, TCP) అలాగే పని చేస్తాయి. అందుకే నీ same React app WiFi లోనూ, mobile data లోనూ పని చేస్తుంది.
4. **Easy debugging** — ఏదైనా fail అయితే, ఏ layer లో fail అయిందో గుర్తించి అక్కడే fix చేయొచ్చు.

నీ MERN connection: నువ్వు `fetch()` రాసేటప్పుడు HTTP (Layer 7) లో ఉంటావు. TCP (Layer 4) reliability చూసుకుంటుంది, IP (Layer 3) routing చూసుకుంటుంది, Ethernet/WiFi (Layer 1-2) actual bits పంపుతుంది. **నువ్వు పై layer లో కూర్చుని, కింద అన్నీ automatic గా జరుగుతాయి** — అదే layering యొక్క magic. తర్వాతి topic లో ఈ layers ని deep గా చూద్దాం.

Key Points

- **Network = 2+ computers data పంచుకోవడానికి connect అయ్యి ఉండటం.** ప్రతి `fetch()` ఒక network call.
- **Protocol = మాట్లాడే rules set** (common language). HTTP, TCP, IP అన్నీ protocols.
- **Packet = data యొక్క చిన్న ముక్క.** పెద్ద data ని packets గా విడగొట్టి పంపి, అవతల మళ్ళీ కలుపుతారు.
- **Size ప్రకారం:** PAN < LAN < MAN < WAN. **Internet = అతిపెద్ద WAN** (network of networks).
- **Topology = devices layout.** Star (ఇళ్ళు/offices — common), Mesh (Internet backbone — fault-tolerant), Bus/Ring (పాతవి).
- **Bandwidth = రోడ్డు వెడల్పు (ఎంత data), Latency = రోడ్డు పొడవు/traffic (ఎంత time).** రెండూ వేరు.
- **Layered architecture = complex పనిని చిన్న independent layers గా విడగొట్టడం.** Modularity, abstraction, interoperability, easy debugging — 4 ప్రయోజనాలు.

Interview దృష్టి

- **"What is the difference between bandwidth and latency?"** — Bandwidth = capacity (Mbps, ఎంత data per second). Latency = delay (ms, ఒక packet చేరడానికి time). Analogy: bandwidth = రోడ్డు ఎన్ని lanes, latency = ఒక car A నుండి B కి చేరే time. High bandwidth ఉన్నా high latency ఉండొచ్చు (satellite internet).
- **"Why do we use layered architecture in networks?"** — Modularity, abstraction, interoperability, easy debugging. ఒక layer మార్చినా మిగతావి పని చేస్తాయి. (పై 4 points చెప్పు.)
- **"LAN vs WAN?"** — LAN = ఒక building, single organization owns, high speed, low latency. WAN = దేశాలు/ప్రపంచం, multiple owners, ఎక్కువ latency. Internet = అతిపెద్ద WAN.
- **"Is the Internet a single network?"** — కాదు. Internet = "network of networks" — కోట్ల LANs/WANs ని కలిపే system, common protocols (TCP/IP) ద్వారా.
- **Gotcha:** "Hub vs Switch" అని అడగొచ్చు (Star topology లో). Hub = అందరికీ broadcast (dumb), Switch = సరైన device కి మాత్రమే పంపుతుంది (smart, MAC address చూసి). Topic 4 లో deep గా చూద్దాం.

---

2. OSI Model — 7 Layers Deep

వివరణ

OSI Model = Open Systems Interconnection model. ఇది ఒక **reference framework** — రెండు computers మాట్లాడే complex పనిని **7 layers** గా విడగొట్టి, ప్రతి layer ఏ పని చేస్తుందో standardize చేస్తుంది. దీన్ని 1984 లో ISO (International Organization for Standardization) రూపొందించింది.

ముఖ్యమైన విషయం: OSI model అనేది **theoretical/conceptual model** — real internet దీన్ని exact గా follow చేయదు (real internet TCP/IP model వాడుతుంది, తర్వాతి topic). కానీ OSI ప్రతి interview లో అడుగుతారు, ఎందుకంటే ఇది **networking ని అర్థం చేసుకోవడానికి ఉత్తమమైన mental map.** ప్రతి protocol (HTTP, TCP, IP, Ethernet) ఏ layer లో ఉంటుందో ఈ model చెప్తుంది.

7 layers ని **పైనుండి కిందకి** (application → physical) చూద్దాం, ఎందుకంటే నువ్వు (developer) పైన ఉంటావు, data కిందకి ప్రయాణించి network లోకి వెళ్తుంది:

```
  ┌──────────────────────────────────────────────┐
7 │  Application    ← నువ్వు ఇక్కడ (HTTP, fetch)  │  "మనుషులకి కనిపించే data"
  ├──────────────────────────────────────────────┤
6 │  Presentation   ← encryption, compression       │  "data format"
  ├──────────────────────────────────────────────┤
5 │  Session        ← connection sessions           │  "సంభాషణ నిర్వహణ"
  ├──────────────────────────────────────────────┤
4 │  Transport      ← TCP/UDP, ports, reliability    │  "end-to-end delivery"
  ├──────────────────────────────────────────────┤
3 │  Network        ← IP, routing, routers           │  "ఏ దారి? (routing)"
  ├──────────────────────────────────────────────┤
2 │  Data Link      ← MAC, switches, Ethernet frames│  "పక్క device కి"
  ├──────────────────────────────────────────────┤
1 │  Physical       ← cables, signals, bits          │  "0s and 1s"
```

```
      ↓ data కిందకి వెళ్తుంది ↓ network cable లోకి
```

Real-life Scenario

> **OSI 7 layers = ఒక company లో ఒక ఉత్తరం పంపే process (CEO నుండి postman దాకా).** నీ company CEO వేరే company CEO కి message పంపాలనుకుంటున్నాడు:
>
> - **Layer 7 (CEO — Application):** "నేను ఈ deal ని confirm చేస్తున్నాను" అనే *అసలు message* ఆలోచిస్తాడు. ఉత్తరం ఎలా వెళ్తుందో అతనికి అనవసరం.
> - **Layer 6 (Translator — Presentation):** Message ని అవతలివారికి అర్థమయ్యే భాషలోకి translate చేస్తాడు, secret code లో రాస్తాడు (encryption).
> - **Layer 5 (Secretary — Session):** ఈ సంభాషణ ని manage చేస్తాడు — "ఇది 3వ ఉత్తరం, గత సంభాషణకి కొనసాగింపు" అని track చేస్తాడు.
> - **Layer 4 (Dispatch manager — Transport):** పెద్ద document ని పేజీలుగా విడగొట్టి (segments), నంబర్లు వేసి, "అన్నీ చేరాయా?" అని ensure చేస్తాడు.
> - **Layer 3 (Post office sorting — Network):** ఏ నగరానికి, ఏ route లో వెళ్ళాలో నిర్ణయిస్తాడు (address చూసి routing).
> - **Layer 2 (Local delivery hub — Data Link):** పక్క hub కి ఎలా చేర్చాలో చూస్తాడు, cover మీద local address వేస్తాడు.
> - **Layer 1 (Truck/road — Physical):** నిజంగా ఉత్తరాన్ని రోడ్డు మీద తీసుకెళ్తుంది.
>
> అవతలి company లో ఇదే process **తలకిందులుగా** (Layer 1 → 7) జరిగి, చివరికి వాళ్ళ CEO కి message చేరుతుంది. **ప్రతి layer తన counterpart తో మాత్రమే "మాట్లాడుతుంది"** (CEO ↔ CEO, translator ↔ translator). ఇదే OSI యొక్క ఆత్మ.

Mnemonic — 7 layers ఎలా గుర్తుంచుకోవాలి

పైనుండి కిందకి (Layer 7 → 1):

> **A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing (Application, Presentation, Session, Transport, Network, Data Link, Physical)

కిందనుండి పైకి (Layer 1 → 7):

> **P**lease **D**o **N**ot **T**hrow **S**ausage **P**izza **A**way (Physical, Data Link, Network, Transport, Session, Presentation, Application)

Telugu trick: "**అ**ందరూ **ప్ర**జలు **సే**వ **ట్రా**న్స్‌పోర్ట్ **నె**ట్ **డే**టా **ఫి**జికల్" — ఏదైనా ఒకటి గట్టిగా పట్టుకో. Interview లో "7 layers చెప్పు" అంటే instant గా చెప్పాలి.

ప్రతి Layer లోతుగా

Layer 7 — Application Layer

- **పని:** Network services ని నేరుగా **end-user application** కి అందించడం. ఇక్కడే HTTP, DNS, FTP, SMTP ఉంటాయి.
- **గమనిక:** ఇది నీ React app కాదు — ఇది నీ app వాడే **protocols** (HTTP). నీ `fetch()` ఇక్కడ మొదలవుతుంది.
- **Data unit:** Message / Data.
- **MERN:** `fetch("https://api.com")` — ఈ HTTP request ఇక్కడ పుడుతుంది.

Layer 6 — Presentation Layer

- **పని:** Data ని **format, encrypt, compress** చేయడం. "Data ఏ రూపంలో ఉంటుంది?" చూసుకుంటుంది.
- **ఉదాహరణలు:** SSL/TLS encryption, JPEG/PNG image formats, JSON/UTF-8 encoding, gzip compression.
- **MERN:** నీ API JSON పంపుతుంది, HTTPS లో TLS encryption జరుగుతుంది — conceptually ఇక్కడ. (Real internet లో TLS ని Layer 4-7 మధ్య పెడతారు.)
- **గుర్తుంచుకో:** "Presentation = data యొక్క **presentation/rూపం**" (encryption, compression, encoding).

Layer 5 — Session Layer

- **పని:** రెండు applications మధ్య **session** (సంభాషణ) ని open, manage, close చేయడం. "ఎవరు ఎప్పుడు మాట్లాడాలి, session ఎంతసేపు ఉండాలి."
- **ఉదాహరణలు:** Login sessions, API tokens conceptually, RPC, checkpoints (పెద్ద file transfer మధ్యలో ఆగితే మళ్ళీ అక్కడినుండి).
- **MERN:** నీ user login చేసినప్పుడు ఒక session ఏర్పడుతుంది — ఆ concept ఇక్కడిది.

Layer 4 — Transport Layer (అత్యంత ముఖ్యం)

- **పని: End-to-end reliable delivery.** Data ని segments గా విడగొట్టడం, ports ద్వారా సరైన application కి చేర్చడం, lost packets మళ్ళీ పంపడం, order maintain చేయడం.
- **Protocols: TCP** (reliable, ordered — HTTP వాడేది), **UDP** (fast, unreliable — video/gaming).
- **Data unit:** Segment (TCP) / Datagram (UDP).
- **MERN:** నీ `fetch()` కింద TCP ఇక్కడ పని చేస్తుంది — reliability guarantee ఇస్తుంది. Port 443 (HTTPS) ఇక్కడిదే. Topic 6 లో deep గా.

Layer 3 — Network Layer

- **పని: Routing** — packet ని source నుండి destination కి, వేర్వేరు networks దాటించి, ఏ దారిలో పంపాలో నిర్ణయించడం. **Logical addressing (IP).**
- **Protocols/devices: IP** (IPv4/IPv6), **routers**, ICMP (ping).
- **Data unit:** Packet.
- **MERN:** నీ packet కి "GitHub server IP 140.82.112.3 కి వెళ్ళు" అని address ఇక్కడ పడుతుంది. Routers ఇక్కడ పని చేస్తాయి. Topic 5.

Layer 2 — Data Link Layer

- **పని: పక్క device (same network) కి** frame ని చేర్చడం. **Physical addressing (MAC).** Error detection (CRC). Switches ఇక్కడ.
- **Protocols/devices: Ethernet, WiFi (802.11), switches, MAC address, ARP.**
- **Data unit:** Frame.
- **MERN:** నీ WiFi router కి packet చేర్చడం ఇక్కడ. Topic 4.

Layer 1 — Physical Layer

- **పని:** అసలు **0s and 1s ని physical signals** (electrical/light/radio) గా మార్చి cable/air ద్వారా పంపడం.
- **ఉదాహరణలు:** Cables (Ethernet, fiber optic), radio waves (WiFi), voltage levels, hubs, repeaters.
- **Data unit:** Bit.
- **MERN:** నీ data చివరికి WiFi radio waves గా, లేదా fiber cable లో light pulses గా మారుతుంది — ఇక్కడ.

సంక్షిప్త Table — ఒక్క చూపులో OSI

Layer	పేరు	పని (ఒక్క మాటలో)	Data Unit	ఉదాహరణలు (protocols/devices)
7	Application	User services	Data/Message	HTTP, DNS, FTP, SMTP, WebSocket
6	Presentation	Format/encrypt/compress	Data	TLS/SSL, JPEG, JSON, gzip, UTF-8
5	Session	Session manage	Data	Sockets, RPC, login sessions
4	Transport	Reliable delivery, ports	**Segment**	**TCP, UDP**
3	Network	Routing, IP addressing	**Packet**	**IP, routers, ICMP**
2	Data Link	పక్క device కి, MAC	**Frame**	**Ethernet, WiFi, switch, ARP**
1	Physical	Bits → signals	**Bit**	Cables, fiber, hubs, radio

Data unit పేర్లు గుర్తుంచుకో (interview గా అడుగుతారు): Layer 4 = **Segment**, Layer 3 = **Packet**, Layer 2 = **Frame**, Layer 1 = **Bit**. Trick: "పైనుండి **SPFB** — Segment, Packet, Frame, Bit." వీటిని కలిపి **PDU (Protocol Data Unit)** అంటారు.

ఎవరు ఏ layer వాడతారు — Devices mapping

```
Layer 7  ──►  నీ browser, server apps
Layer 4  ──►  (software: OS kernel TCP/IP stack)
Layer 3  ──►  Router (IP చూసి route చేస్తుంది)
Layer 2  ──►  Switch (MAC చూసి పంపుతుంది), NIC (network card)
Layer 1  ──►  Hub, repeater, cable, WiFi antenna
```

గుర్తుంచుకో: Router = Layer 3 (IP), **Switch = Layer 2** (MAC), **Hub = Layer 1** (dumb, అందరికీ broadcast). ఇది interview favorite. "Router ఏ layer?" → Layer 3. "Switch?" → Layer 2.

Key Points

- **OSI = 7-layer conceptual reference model** (theoretical). Real internet TCP/IP వాడుతుంది, కానీ OSI అర్థం చేసుకోవడానికి best map.
- **7 layers (పైనుండి):** Application, Presentation, Session, Transport, Network, Data Link, Physical. Mnemonic: **A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing.
- **ప్రతి layer తన counterpart తోనే logically మాట్లాడుతుంది** (TCP ↔ TCP), కానీ data actual గా అన్ని layers గుండా కిందకి-పైకి ప్రయాణిస్తుంది.
- **Data units:** Segment (L4) → Packet (L3) → Frame (L2) → Bit (L1).
- **Device mapping:** Router = L3, Switch = L2, Hub = L1. TCP/UDP = L4, IP = L3, Ethernet/MAC = L2.
- **గుర్తుంచుకోవాల్సిన top-4:** Layer 4 Transport (TCP/UDP), Layer 3 Network (IP), Layer 2 Data Link (MAC), Layer 1 Physical — ఇవి interview లో 90% అడిగేవి.

Interview దృష్టి

- **"Explain the OSI model / 7 layers."** — Mnemonic తో 7 layers, ప్రతి దాని పని ఒక్క వాక్యంలో, ఒక example protocol. పైనుండి కిందకి చెప్పు.
- **"Which layer does [X] belong to?"** — Fast recall: HTTP=7, TLS=6 (conceptually), TCP/UDP=4, IP=4? కాదు **IP=3**, Ethernet/MAC=2, cable=1. Router=3, Switch=2.
- **"Difference between OSI and TCP/IP model?"** — తర్వాతి topic. Short: OSI=7 layers theoretical, TCP/IP=4 layers practical (real internet).
- **"What is a PDU?"** — Protocol Data Unit — ప్రతి layer లో data కి పేరు. Segment/Packet/Frame/Bit.
- **Gotcha — "OSI ని real internet వాడుతుందా?"** — లేదు. OSI reference/teaching model. Real internet TCP/IP model follow చేస్తుంది. కానీ engineers OSI terminology (layer 3, layer 7 load balancer) రోజూ వాడతారు.
- **Gotcha — Presentation & Session layers:** Real TCP/IP లో ఇవి separate కాదు — application layer లో కలిసిపోతాయి. కానీ OSI లో వేరు. "TLS ఏ layer?" — OSI ప్రకారం 6, practically application-level.

---

3. TCP/IP Model (vs OSI, Encapsulation/Decapsulation)

వివరణ

TCP/IP Model = internet నిజంగా వాడే practical model. OSI 7 layers theoretical అయితే, TCP/IP అనేది real-world లో పని చేసే **4-layer (కొన్ని books 5-layer)** model. దీన్ని 1970s లో US Defense (DARPA) రూపొందించింది — internet దీని మీదే నడుస్తుంది. అందుకే దీని పేరు internet లో అత్యంత ముఖ్యమైన రెండు protocols — **TCP** (transport) + **IP** (network) — పేరుతో వచ్చింది.

ఎందుకు రెండు models? OSI ముందు ఒక perfect theoretical design గా వచ్చింది. కానీ ఆచరణలో internet అప్పటికే TCP/IP మీద పని చేస్తోంది, అది simpler. కాబట్టి **OSI = teaching/reference కి, TCP/IP = actual implementation కి.** ఇద్దరూ same idea (layering), కానీ layer counts వేరు.

OSI vs TCP/IP — Mapping

TCP/IP, OSI యొక్క కొన్ని layers ని కలిపేస్తుంది:

```
   OSI (7 layers)              TCP/IP (4 layers)

┌──────────────────┐        ┌───────────────────────┐
│ 7 Application    │  ┐     │                       │
├──────────────────┤  │     │                       │
│ 6 Presentation   │  ├──►  │  Application          │  (HTTP, DNS, TLS, FTP)
├──────────────────┤  │     │                       │
│ 5 Session        │  ┘     │                       │
├──────────────────┤        ├───────────────────────┤
│ 4 Transport      │  ────► │  Transport            │  (TCP, UDP)
├──────────────────┤        ├───────────────────────┤
│ 3 Network        │  ────► │  Internet             │  (IP, ICMP, routers)
├──────────────────┤        ├───────────────────────┤
│ 2 Data Link      │  ┐     │                       │
├──────────────────┤  ├──►  │  Network Access       │  (Ethernet, WiFi, MAC)
│ 1 Physical       │  ┘     │  (a.k.a. Link)        │
└──────────────────┘        └───────────────────────┘
```

అంశం	OSI Model	TCP/IP Model
Layers	7	4 (కొన్ని books 5)
రకం	Theoretical / reference	Practical / implemented
వాడకం	నేర్చుకోవడానికి, terminology కి	Real internet నడిచేది దీని మీద
Application layer	3 layers (5,6,7 విడిగా)	1 layer (అన్నీ కలిపి)
Network access	2 layers (1,2 విడిగా)	1 layer (Link) కలిపి
ఎవరు develop చేశారు	ISO	DARPA (US Defense)
Protocol dependency	Protocol-independent (general)	TCP/IP protocols చుట్టూ built

గుర్తుంచుకో: TCP/IP model లో top 3 OSI layers (App, Presentation, Session) → ఒక్క **Application** layer అయ్యాయి. Bottom 2 (Data Link, Physical) → ఒక్క **Network Access/Link** layer అయ్యాయి. Middle రెండు (Transport, Network) same గా ఉన్నాయి. **అందుకే HTTP, TLS, DNS అన్నీ "application layer" అని మనం రోజూ అంటాం** — TCP/IP prspective.

Real-life Scenario

> **OSI vs TCP/IP = వంటల recipe book vs నిజంగా వంట చేయడం.**
>
> - **OSI** = ఒక perfect, detailed recipe book — ప్రతి step ని 7 sub-steps గా విడగొట్టి రాసినది. Theory perfect, నేర్చుకోవడానికి బాగుంది.
> - **TCP/IP** = నిజంగా ఒక అనుభవం ఉన్న వంటవాడు వండే విధానం — కొన్ని steps ని కలిపేసి, 4 steps లో practical గా చేస్తాడు. అదే ఆహారం, కానీ efficient.
>
> నీ hotel (internet) నిజంగా నడిచేది వంటవాడి practical విధానం (TCP/IP) మీదే. కానీ కొత్తవాళ్ళకి నేర్పేటప్పుడు detailed recipe book (OSI) వాడతారు. **అందుకే interview లో రెండూ తెలియాలి.**

Encapsulation — Data కిందకి ప్రయాణం (అత్యంత ముఖ్యం)

ఇది CN లో అత్యంత important concepts లో ఒకటి. **నీ data ప్రతి layer గుండా కిందకి వెళ్తున్నప్పుడు, ప్రతి layer దానికి తన సొంత header (కొన్నిసార్లు trailer) ని జోడిస్తుంది.** ఈ "header జోడించడం" = **Encapsulation.**

> **Analogy — ఉత్తరాన్ని పొరలుగా cover చేయడం (Russian dolls / gift wrapping).** నీ data ఒక చిన్న చీటీ. దాన్ని పంపడానికి:
>
> - Transport layer దాన్ని ఒక cover లో పెట్టి "port number" రాస్తుంది.
> - Network layer ఆ cover ని ఇంకో పెద్ద cover లో పెట్టి "IP address" రాస్తుంది.
> - Data Link layer దాన్ని మరో cover లో పెట్టి "MAC address" రాస్తుంది.
> - **ప్రతి layer ఒక కొత్త cover (header) చుడుతుంది** — లోపలిది ఏమిటో పట్టించుకోకుండా.

```
నీ data (HTTP request): "GET /users"
         │
         ▼  Application Layer
┌───────────────────────────────────┐
│  HTTP Header │  "GET /users"       │   ← Application data
└───────────────────────────────────┘
         │
         ▼  Transport Layer (TCP) — port జోడిస్తుంది
┌───────────┬───────────────────────────────┐
│ TCP Header│  HTTP Header │ "GET /users"     │    = SEGMENT
│ (port 443)│                                 │
└───────────┴───────────────────────────────┘
         │
         ▼  Network Layer (IP) — IP address జోడిస్తుంది
┌──────────┬───────────┬──────────────────────────┐
│ IP Header│ TCP Header│  HTTP data ...            │    = PACKET
│(dest IP) │           │                           │
└──────────┴───────────┴──────────────────────────┘
         │
         ▼  Data Link Layer (Ethernet) — MAC జోడిస్తుంది (header + trailer)
┌───────────┬──────────┬───────────┬─────────────┬────────┐
│Eth Header │ IP Header│ TCP Header│  HTTP data   │Eth CRC │  = FRAME
│(dest MAC) │          │           │              │(trailer)│
└───────────┴──────────┴───────────┴─────────────┴────────┘
```

```
        |
        ▼  Physical Layer
  0101110100101...  ← BITS (cable/WiFi లోకి)
```

గమనించు: కిందకి వెళ్ళే కొద్దీ headers **పెరుగుతూ** ఉంటాయి (గుడ్డు చుట్టూ పొరలు). నీ actual HTTP data లోపల unchanged గా ఉంటుంది. Data Link layer మాత్రమే **trailer** (CRC error-check) కూడా జోడిస్తుంది — అది topic 4 లో.

Decapsulation — అవతల reverse

Destination computer కి data చేరాక, **తలకిందుల process** జరుగుతుంది — ప్రతి layer తన header ని తీసేసి (peel), లోపలిది పై layer కి పంపుతుంది:

```
  BITS వస్తాయి (Physical)
        | ▲
        ▼ |  Data Link: Eth header/trailer తీసేసి, MAC check → IP packet పైకి
        ▼ |  Network:   IP header తీసేసి, "నా IP నేనేనా?" check → TCP segment పైకి
        ▼ |  Transport: TCP header తీసేసి, port చూసి సరైన app కి → HTTP data పైకి
        ▼ |  Application: HTTP request server కి అందుతుంది → "GET /users" process
```

అంటే: **Encapsulation (sender side, top→down, headers జోడించడం)** → **network ప్రయాణం** → **Decapsulation (receiver side, bottom→up, headers తీసేయడం).** ఇదే ప్రతి `fetch()` లో జరుగుతుంది.

MERN connection — నీ fetch() లో ఇది ఎలా జరుగుతుంది

```
fetch("https://api.github.com/users/torvalds")
```

- **Application:** Browser ఒక HTTP GET request తయారు చేస్తుంది (`GET /users/torvalds HTTP/1.1\nHost: api.github.com`).
- **Transport (TCP):** OS దాన్ని segments గా విడగొట్టి, source port (random, ఉదా 52000) + dest port (443) header జోడిస్తుంది.
- **Network (IP):** నీ IP (source) + GitHub IP 140.82.x.x (dest) header జోడిస్తుంది.
- **Link (Ethernet/WiFi):** నీ WiFi router MAC address header జోడించి, bits గా మార్చి WiFi radio waves గా పంపుతుంది.
- GitHub server లో ఇదంతా **reverse (decapsulation)** జరిగి, వాళ్ళ app "GET /users/torvalds" అందుకుంటుంది.

ఇదంతా నీకు కనిపించదు — నువ్వు రాసేది ఒక్క `fetch()` line. కిందున్న 4 layers అంతా OS + hardware చేస్తాయి. **అదే layering + encapsulation యొక్క power.**

Key Points

- **TCP/IP model = internet నిజంగా వాడేది** (practical, 4 layers). OSI = teaching/reference (theoretical, 7 layers).
- **4 layers:** Application (HTTP/DNS/TLS), Transport (TCP/UDP), Internet (IP), Network Access/Link (Ethernet/WiFi).
- **Mapping:** OSI top-3 → TCP/IP Application; OSI bottom-2 → TCP/IP Link; Transport & Network same.

- **Encapsulation = sender side, top→down, ప్రతి layer తన header జోడిస్తుంది.** Data unit పెరుగుతూ: Data → Segment → Packet → Frame → Bits.
- **Decapsulation = receiver side, bottom→up, ప్రతి layer తన header తీసేస్తుంది.**
- **Data Link layer మాత్రమే trailer (CRC) కూడా జోడిస్తుంది** (header + trailer). మిగతావి header only.

Interview దృష్టి

- **"OSI vs TCP/IP model?"** — OSI 7 theoretical layers, TCP/IP 4 practical. TCP/IP internet reality. Table లోని 2-3 differences చెప్పు (layers, practical vs theoretical, application layer combining).
- **"Explain encapsulation."** — Top→down, ప్రతి layer తన header జోడిస్తుంది; Data→Segment(TCP header)→Packet(IP header)→Frame(Ethernet header+trailer)→Bits. Receiver decapsulates (reverse). **Diagram గీయగలగాలి** — interview లో often అడుగుతారు గీయమని.
- **"Which model does the internet use?"** — TCP/IP. కానీ engineers OSI terminology (L3/L7) రోజూ మాట్లాడతారు.
- **"What header does each layer add?"** — Transport → port numbers; Network → IP addresses; Data Link → MAC addresses + CRC trailer.
- **Gotcha:** "5-layer model" — కొన్ని textbooks TCP/IP ని 5 layers గా చూపిస్తాయి (Physical + Data Link విడిగా). 4 vs 5 debate — నువ్వు "commonly 4, some split link into physical+data-link making 5" అని చెప్పు.
- **Gotcha:** Encapsulation ≠ encryption. Encapsulation = headers wrapping (అన్ని packets కి). Encryption = data ని secret code లోకి మార్చడం (TLS, ప్రత్యేకం).

Part 2 — Layers (ప్రతి layer లోతుగా)

> ఇప్పుడు మనం model తెలుసుకున్నాం. ఈ Part లో **కింది 4 layers ని (Physical → Application) ఒక్కొక్కటిగా లోతుగా** చూద్దాం. ప్రతి layer లో: ఏ addressing (MAC vs IP vs port), ఏ devices (switch vs router), ఏ protocols (Ethernet, IP, TCP/UDP), ఏ problems solve చేస్తుంది. ఇదే CN యొక్క గుండె. ఇక్కడ subnetting math, TCP handshake లాంటి interview-favorite topics ఉన్నాయి — గట్టిగా చదువు.

---

4. Physical & Data Link Layer

వివరణ

ఇవి OSI యొక్క **అట్టడుగు రెండు layers** — data నిజంగా "waya" మీద ప్రయాణించే చోటు. వీటిని కలిపి చూద్దాం ఎందుకంటే TCP/IP model లో ఇవి ఒక్క **Link layer** గా ఉంటాయి.

Physical Layer (Layer 1): అసలు **bits (0s and 1s) ని physical signals గా మార్చి పంపడం.** ఇది "ఎలా" data travel చేస్తుందో చూస్తుంది:

- **Copper cable (Ethernet):** bits → **electrical voltage** (high voltage = 1, low = 0).
- **Fiber optic:** bits → **light pulses** (light on = 1, off = 0). అత్యంత fast, long distance.
- **WiFi:** bits → **radio waves** (వేర్వేరు frequencies).

Physical layer కి "meaning" తెలియదు — దానికి తెలిసింది voltage/light పంపడం మాత్రమే. Devices: cables, hubs, repeaters, NIC (network card యొక్క physical భాగం).

Data Link Layer (Layer 2): Physical layer పైన కూర్చుని, **పక్క device కి (same local network లో) reliable గా frame ని చేర్చడం.** ఇక్కడ 3 కీలక concepts: **MAC address, switches, error detection (CRC).**

Real-life Scenario

> **Physical vs Data Link = రోడ్డు vs local postman.**
>
> - **Physical layer = రోడ్డు, vehicle, petrol.** నిజంగా ఉత్తరాన్ని మోసుకెళ్ళే భౌతిక మార్గం. రోడ్డుకి ఉత్తరంలో ఏముందో తెలియదు — అది కేవలం transport.
> - **Data Link layer = నీ colony postman.** అతనికి **నీ colony లోని ప్రతి ఇంటి door number (MAC address)** తెలుసు. "ఈ ఉత్తరం 3వ ఇంటికి" అని local గా చేరుస్తాడు. కానీ వేరే నగరానికి ఎలా వెళ్ళాలో అతనికి తెలియదు — అది post office (router, Layer 3) పని.
>
> **ముఖ్యమైన తేడా:** postman **local delivery** మాత్రమే చేస్తాడు (same network). వేరే నగరం (వేరే network) కి కావాలంటే, ముందు అది central post office (gateway/router) కి వెళ్ళాలి. ఇదే MAC (local) vs IP (global) తేడా.

MAC Address — physical/hardware address

MAC (Media Access Control) address = ప్రతి network device కి ఉన్న unique, permanent, hardware-level address. ఇది device తయారైనప్పుడే NIC (network card) లో burn అవుతుంది — ఇంచుమించు మారదు.

```
MAC Address ఉదాహరణ:   A4:C3:F0:85:AC:2D
                      └──┬──┘ └──┬──┘
                    తయారీ కంపెనీ  device unique
                    (OUI, 24 bits) (24 bits)
   మొత్తం 48 bits = 6 bytes, hexadecimal లో రాస్తారు
```

- **48 bits (6 bytes),** hexadecimal (A4:C3:F0:85:AC:2D).
- మొదటి 3 bytes = **OUI** — తయారీ కంపెనీ (Apple, Intel...). చివరి 3 bytes = ఆ కంపెనీ ఇచ్చిన unique number.
- **Permanent + physical** — device తో పాటే వస్తుంది. (IP address మారుతుంది, MAC మారదు — పోలిక కింద.)

MAC vs IP — అత్యంత ముఖ్యమైన పోలిక (interview favorite):

అంశం	MAC Address	IP Address
Layer	Data Link (Layer 2)	Network (Layer 3)
పొడవు	48 bits (6 bytes)	IPv4: 32 bits, IPv6: 128 bits
రూపం	A4:C3:F0:85:AC:2D	192.168.1.5
మారుతుందా?	Permanent (hardware లో burn)	మారుతుంది (network మారితే మారుతుంది)
పరిధి (scope)	**Local network మాత్రమే**	**Global (internet అంతా)**
ఎవరు ఇస్తారు	Manufacturer	ISP / DHCP / admin
analogy	నీ **Aadhaar number** (permanent, నీది)	నీ **ప్రస్తుత address** (ఇల్లు మారితే మారుతుంది)

కీలక intuition: MAC = **ఎవరు (who — permanent identity)**, IP = **ఎక్కడ (where — current location)**. Data వేరే network కి వెళ్ళాలంటే IP అవసరం (global). Same network లో పక్క device కి చేరాలంటే MAC అవసరం (local). రెండూ కలిసి పని చేస్తాయి — ఇదే ARP (కింద).

ARP — IP ని MAC గా మార్చే protocol

సమస్య: నీ computer కి destination **IP address** తెలుసు (ఉదా, నీ router 192.168.1.1). కానీ actual frame పంపాలంటే దాని **MAC address** కావాలి (Data Link layer MAC తో పని చేస్తుంది). ఈ IP → MAC translation ని **ARP (Address Resolution Protocol)** చేస్తుంది.

```
నీ computer:  "192.168.1.1 అనే IP ఎవరిది? మీ MAC address చెప్పండి!"
               ──────────► BROADCAST (network లో అందరికీ)

Router:        "అది నేనే! నా MAC address A4:C3:F0:85:AC:2D"
               ◄───────── UNICAST (నీకు మాత్రమే reply)

నీ computer:  ఇప్పుడు ARP cache లో save: 192.168.1.1 → A4:C3:F0:85:AC:2D
```

- **ARP request = broadcast** ("ఈ IP ఎవరిది?" అందరినీ అడుగుతుంది).
- **ARP reply = unicast** (సరైన device మాత్రమే "నేనే, ఇదిగో నా MAC" అని చెప్తుంది).
- ఫలితం **ARP cache** లో save అవుతుంది (మళ్ళీ మళ్ళీ అడగకుండా). Terminal లో `arp -a` చేస్తే చూడొచ్చు.

Switch vs Hub — Layer 2 vs Layer 1

Hub (Layer 1 — dumb): ఏ port లో data వచ్చినా, దాన్ని **అన్ని ports కి broadcast** చేస్తుంది. అందరికీ పంపేస్తుంది, సరైన device ఏదో పట్టించుకోదు → traffic waste, collisions, insecure. ఇప్పుడు దాదాపు వాడరు.

Switch (Layer 2 — smart): MAC address table maintain చేస్తుంది — "ఏ MAC address ఏ port కి connected" అని నేర్చుకుంటుంది. Data వస్తే, destination MAC చూసి **సరైన port కి మాత్రమే** పంపుతుంది. Efficient, secure, no unnecessary broadcast.

```
HUB (dumb):                     SWITCH (smart):
  data వచ్చింది                    data వచ్చింది (dest MAC: B)
       │                              │
   ┌───┼───┐                       MAC table చూస్తుంది:
   ▼   ▼   ▼                        A→port1, B→port2, C→port3
  A   B   C                           │
  అందరికీ! (waste)                     ▼ port2 కి మాత్రమే
                                        B (సరైన device)
```

Error Detection — CRC (Cyclic Redundancy Check)

సమస్య: Physical layer లో electrical/radio signals — noise వల్ల ఒక bit flip అవ్వొచ్చు (0 → 1). ఈ error ని ఎలా పట్టుకోవాలి?

పరిష్కారం: CRC. Sender data మీద ఒక **mathematical calculation (polynomial division)** చేసి, ఒక చిన్న **checksum** ని frame trailer లో జోడిస్తుంది. Receiver అదే calculation చేసి, తన checksum sender checksum తో match అవుతుందా చూస్తుంది.

- **Match అయితే:** data safe (no error).
- **Match కాకపోతే:** frame corrupt అయ్యింది → **discard** (పడేస్తారు; TCP layer దాన్ని మళ్ళీ పంపమని అడుగుతుంది).

> **Analogy — bill total verification.** నువ్వు 10 items కొన్నావు. Cashier వాటి prices రాసి, చివర్లో **total** వేస్తాడు. ఇంటికెళ్ళి నువ్వు మళ్ళీ 10 prices కూడి total తో match చేస్తావు. Match కాకపోతే "ఏదో తప్పు జరిగింది" అని తెలుస్తుంది. **CRC = ఆ total.** ఏ item తప్పో చెప్పదు, కానీ "ఎక్కడో తప్పుంది" అని కచ్చితంగా చెప్తుంది.

గుర్తుంచుకో: CRC **error detection** మాత్రమే (తప్పు ఉందా లేదా). **Error correction** కాదు (తప్పు ఎక్కడ, ఎలా fix). Data Link layer error detect చేసి frame పడేస్తుంది, correction (retransmission) TCP (Layer 4) చేస్తుంది.

Ethernet & CSMA/CD — collision నివారణ

Ethernet = అత్యంత common wired LAN technology (Data Link + Physical). పాత Ethernet లో అందరూ ఒకే cable share చేసేవారు (bus topology). అప్పుడు **collision problem:** ఇద్దరు devices ఒకేసారి data పంపితే, signals ఢీకొని (collide) రెండూ corrupt అవుతాయి.

పరిష్కారం — CSMA/CD (Carrier Sense Multiple Access / Collision Detection):

```
1. Carrier Sense:  "cable busy గా ఉందా?" ముందు వింటుంది (listen before talk)
2. Multiple Access: cable free అయితే, ఎవరైనా పంపొచ్చు
3. Collision Detection: పంపేటప్పుడు collision జరిగితే detect చేస్తుంది
4. Backoff: collision అయితే, ఇద్దరూ RANDOM time ఆగి మళ్ళీ ప్రయత్నిస్తారు
```

> **Analogy — group conversation లో మాట్లాడటం.** ఒక గదిలో అందరూ కూర్చున్నారు (shared cable). నువ్వు మాట్లాడేముందు — (1) ఎవరైనా మాట్లాడుతున్నారా అని **వింటావు** (carrier sense). (2) Silent అయితే **మాట్లాడతావు**. (3) పొరపాటున ఇద్దరూ ఒకేసారి మొదలుపెడితే (**collision**), ఇద్దరూ ఆగిపోతారు. (4) ఇద్దరూ **random time** ఆగి (ఒకరు 2 sec, ఒకరు 5 sec) మళ్ళీ మొదలుపెడతారు — random కాబట్టి మళ్ళీ collide అయ్యే chance తక్కువ. ఇదే CSMA/CD.

గమనిక: ఇవాళ్టి Ethernet **switches** వాడుతుంది (shared cable కాదు, ప్రతి device కి dedicated link) — కాబట్టి collisions దాదాపు లేవు, CSMA/CD ఇప్పుడు legacy. కానీ **WiFi** లో ఇలాంటిదే **CSMA/CA** (CA = Collision **Avoidance**) వాడతారు — WiFi లో collision detect చేయడం కష్టం కాబట్టి, ముందే avoid చేస్తారు.

Half-Duplex vs Full-Duplex

- **Half-Duplex:** ఒకేసారి ఒక దిశలో మాత్రమే (walkie-talkie — ఒకరు మాట్లాడితే ఇంకొకరు వినాలి). పాత hub-based Ethernet.
- **Full-Duplex:** రెండు దిశల్లో ఏకకాలంలో (phone call — ఇద్దరూ ఒకేసారి మాట్లాడొచ్చు). Modern switch-based Ethernet. Collision లేదు (dedicated send/receive paths).

Collision Domain vs Broadcast Domain (interview)

- **Collision Domain:** collisions జరగగల devices set. **Hub = అన్ని ports ఒకే collision domain** (అందరూ collide అవ్వొచ్చు). **Switch = ప్రతి port వేరే collision domain** (collisions తగ్గుతాయి — switch యొక్క ప్రధాన లాభం).
- **Broadcast Domain:** broadcast (అందరికీ) message చేరే devices set. **Switch broadcast ని forward చేస్తుంది** (ఒకే broadcast domain). **Router broadcast ని ఆపుతుంది** (ప్రతి interface వేరే broadcast domain).
- **VLAN (Virtual LAN):** ఒక physical switch ని logically అనేక వేర్వేరు networks (broadcast domains) గా విడగొట్టడం. ఉదా HR, Engineering ని same switch మీద కానీ separate VLANs — traffic isolate, security. "ఒక switch, అనేక logical LANs."

సారాంశం: Switch collision domains ని విడగొడుతుంది కానీ broadcast domain ఒకటే. Router broadcast domains ని విడగొడుతుంది. ఇది "switch vs router" తేడాని లోతుగా చూపిస్తుంది.

Key Points

- **Physical layer (L1) = bits → physical signals** (voltage/light/radio). Devices: cables, hubs, repeaters.
- **Data Link layer (L2) = పక్క device కి frame చేర్చడం, MAC addressing, error detection.** Devices: switch, NIC.
- **MAC address = 48-bit permanent hardware address** (`A4:C3:F0:85:AC:2D`). **MAC = who (identity), IP = where (location).** MAC local, IP global.

- **ARP = IP → MAC translation.** Request = broadcast, reply = unicast, ఫలితం ARP cache లో.
- **Switch (L2, MAC table, smart, సరైన port కి) vs Hub (L1, broadcast, dumb).**
- **CRC = error detection** (checksum, polynomial). Detect మాత్రమే, correct కాదు. Corrupt frame discard.
- **CSMA/CD = Ethernet collision handling** (sense → send → detect collision → random backoff). WiFi = CSMA/CA (avoidance).

Interview దృష్టి

- **"MAC vs IP address?"** — MAC = L2, 48-bit, permanent, hardware, local scope. IP = L3, 32/128-bit, changeable, logical, global scope. **MAC = identity (who), IP = location (where).** (Table remember.)
- **"What is ARP?"** — IP address తెలుసు కానీ MAC కావాలి → ARP resolves IP to MAC. Broadcast request, unicast reply, cached.
- **"Switch vs Hub vs Router?"** — Hub = L1 broadcast (dumb). Switch = L2, MAC table, correct port (smart). Router = L3, IP, వేర్వేరు networks connect. (ఈ మూడు తేడా చాలా అడుగుతారు.)
- **"How does error detection work at Data Link layer?"** — CRC checksum trailer. Receiver recomputes; mismatch → discard frame. Detection only (correction = higher layer/TCP retransmit).
- **"What is CSMA/CD?"** — Shared medium లో collision handling. Sense before send, detect collision, random backoff retry. CD = Ethernet, CA = WiFi.
- **Gotcha:** "Error detection vs correction" — CRC detects, doesn't correct. Retransmission (TCP) fixes. Hamming code లాంటివి correct చేయగలవు కానీ networks లో సాధారణంగా detect+retransmit approach.
- **Gotcha:** WiFi ఎందుకు CSMA/CA (CD కాదు)? — Wireless లో transmit చేస్తూ receive చేయడం కష్టం, collision detect చేయలేరు కాబట్టి ముందే avoid చేస్తారు (RTS/CTS).

---

5. Network Layer — IP, Subnetting, Routing, NAT, DHCP

వివరణ

Network Layer (Layer 3) = packet ని ఒక network నుండి వేరే network కి, routers గుండా, source నుండి destination కి చేర్చడం. Data Link layer local (పక్క device) అయితే, Network layer **global** — internet అంతటా, ఏ దారిలో వెళ్ళాలో (routing) నిర్ణయిస్తుంది. దీని రెండు కీలక పనులు: **(1) logical addressing (IP), (2) routing (path selection).**

ఇది CN లో అత్యంత content-heavy topic — IP addresses, subnetting math, NAT, DHCP అన్నీ ఇక్కడే. నెమ్మదిగా చదువు.

Real-life Scenario

> **Network layer = దేశవ్యాప్త postal system యొక్క sorting & routing.** నీ ఉత్తరం Hyderabad నుండి Delhi వెళ్ళాలి. నీ colony postman (Data Link, topic 4) దాన్ని local post office దాకా మాత్రమే తీసుకెళ్తాడు. అక్కడి నుండి —
>
> - ప్రతి ఉత్తరం మీద **పూర్తి address (PIN code = IP address)** ఉంటుంది — ఏ నగరం, ఏ area.
> - **Sorting centers (routers)** ఆ address చూసి "ఇది ఉత్తరాదికి → ఈ truck ఎక్కించు" అని **next hop** నిర్ణయిస్తాయి.
> - ఏ sorting center కి **మొత్తం route తెలియదు** — కేవలం "తర్వాత ఎక్కడికి పంపాలో" (routing table) తెలుసు. ఉత్తరం hop-by-hop గా Delhi చేరుతుంది.
> - నీ ఇంటి address (private IP) కి బదులు, బయటి ప్రపంచం కోసం నీ ఊరి **main post office address (public IP)** వాడతారు — ఇదే **NAT.**
>
> **Network layer = ఈ PIN-code system + sorting centers.** IP = PIN code (ఎక్కడ), routers = sorting centers (ఏ దారి), NAT = private↔public address mapping. ఇప్పుడు ప్రతి భాగం చూద్దాం.

IP Address — logical/global address

IP (Internet Protocol) address = ప్రతి device కి network లో ఇచ్చే logical address. ఇది "నీ ప్రస్తుత location" — network మారితే మారుతుంది.

IPv4 = 32 bits, 4 octets (bytes) గా, dot తో విడదీసి:

```
192   .  168   .  1   .  5
└─┬─┘    └─┬─┘    └┬┘   └┬┘
8 bits   8 bits  8 bits 8 bits   =  మొత్తం 32 bits

binary లో: 11000000.10101000.00000001.00000101
ప్రతి octet 0–255 (2^8 = 256 విలువలు)
```

- మొత్తం address space = 2^{32} ≈ **4.3 billion** addresses. ఇది చాలదు అయ్యింది (అందుకే IPv6, NAT వచ్చాయి).
- ప్రతి octet **0 నుండి 255** వరకు (8 bits = 256 combinations).

IPv4 Address Classes

పాత రోజుల్లో IP addresses ని **classes** గా విడగొట్టారు (ఇప్పుడు CIDR వాడతారు కానీ interview కి classes తెలియాలి):

Class	మొదటి octet range	Default subnet mask	Networks	Hosts/network	వాడకం
A	1 – 126	255.0.0.0 (/8)	126	~16 million	చాలా పెద్ద orgs
B	128 – 191	255.255.0.0 (/16)	~16K	~65K	Medium orgs
C	192 – 223	255.255.255.0 (/24)	~2M	254	చిన్న networks (ఇళ్ళు)
D	224 – 239	—	—	—	Multicast
E	240 – 255	—	—	—	Reserved (research)

గమనిక: 127.x.x.x = **loopback** (127.0.0.1 = `localhost` — నీ own machine). నీ Node server `localhost:3000` run చేసేటప్పుడు వాడేది ఇదే! ఆ traffic network లోకి వెళ్ళదు, నీ machine లోపలే తిరుగుతుంది.

Private IP ranges (internet లో route అవ్వవు, LAN లో మాత్రమే):

- `10.0.0.0 – 10.255.255.255` (Class A private)
- `172.16.0.0 – 172.31.255.255` (Class B private)
- `192.168.0.0 – 192.168.255.255` (Class C private) — **నీ ఇంటి WiFi ఇదే** (192.168.1.x).

Subnetting + CIDR — worked math (interview must-know)

Subnetting = ఒక పెద్ద network ని చిన్న subnetworks (subnets) గా విడగొట్టడం. ఎందుకు? IP addresses waste అవ్వకుండా, security కోసం, traffic manage కోసం.

Subnet Mask = ఏ భాగం "network", ఏ భాగం "host" అని చెప్పే 32-bit number. **CIDR notation** (`/24`) = ఎన్ని bits network కి అని short గా.

```
IP:          192.168.1.5
Subnet mask: 255.255.255.0   =  /24  (మొదటి 24 bits network)

binary:
IP:     11000000.10101000.00000001 . 00000101
mask:   11111111.11111111.11111111 . 00000000
        └──────── NETWORK (24) ────┘ └HOST(8)┘
```

- **Network bits (1s):** ఏ subnet అని identify చేస్తాయి.
- **Host bits (0s):** ఆ subnet లో individual devices.

కీలక సూత్రాలు (గుర్తుంచుకో):

```
Host bits = 32 - prefix (CIDR number)
మొత్తం addresses = 2^(host bits)
Usable hosts = 2^(host bits) - 2   (-2: network address + broadcast address)
```

ఎందుకు −2? ప్రతి subnet లో మొదటి address = **network address** (subnet ని identify చేస్తుంది, device కి ఇవ్వరు), చివరి address = **broadcast address** (అందరికీ పంపడానికి, device కి ఇవ్వరు). కాబట్టి usable hosts = total − 2.

Worked Example 1 — /24 network

```
Given: 192.168.1.0/24
- Prefix = 24 → host bits = 32 − 24 = 8
- Total addresses = 2^8 = 256
- Usable hosts = 256 − 2 = 254
- Network address = 192.168.1.0   (host bits అన్నీ 0)
- Broadcast address = 192.168.1.255 (host bits అన్నీ 1)
- Usable range = 192.168.1.1  నుండి  192.168.1.254
```

Worked Example 2 — /26 (subnetting a /24 into 4 parts)

```
Given: 192.168.1.0/26  (మనం /24 ని 4 subnets గా విడగొడుతున్నాం)
- Prefix = 26 → host bits = 32 − 26 = 6
- Total per subnet = 2^6 = 64 addresses
- Usable hosts = 64 − 2 = 62
- ఎన్ని subnets? borrowed bits = 26 − 24 = 2 → 2^2 = 4 subnets

4 subnets (block size = 64):
```

Subnet	Network address	Usable range	Broadcast
192.168.1.0/26	192.168.1.0	.1 – .62	192.168.1.63
192.168.1.64/26	192.168.1.64	.65 – .126	192.168.1.127
192.168.1.128/26	192.168.1.128	.129 – .190	192.168.1.191
192.168.1.192/26	192.168.1.192	.193 – .254	192.168.1.255

Trick — "block size": block size = 256 − (mask octet value). /26 mask = 255.255.255.**192**, block size = 256 − 192 = **64**. కాబట్టి subnets 0, 64, 128, 192 వద్ద మొదలవుతాయి. ఇది interview లో fast గా చేయడానికి బంగారు trick.

Worked Example 3 — "ఎన్ని hosts కావాలంటే ఏ prefix?"

```
Q: నాకు 100 hosts కావాలి. ఏ subnet mask?
- 2^h − 2 ≥ 100   →  2^h ≥ 102   →  h = 7 (2^7 = 128, 128−2 = 126 ✓)
- Host bits = 7 → prefix = 32 − 7 = /25
- /25 = 255.255.255.128, 126 usable hosts. ✓
```

CIDR reference table (గుర్తుంచుకో):

CIDR	Subnet Mask	Block size	Usable hosts
/24	255.255.255.0	256	254
/25	255.255.255.128	128	126
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30
/28	255.255.255.240	16	14
/29	255.255.255.248	8	6
/30	255.255.255.252	4	2

Worked Example 4 — VLSM (Variable Length Subnet Masking)

Real networks లో అన్ని subnets ఒకే size అవసరం లేదు. **VLSM = ఒక్కో subnet ని అవసరాన్ని బట్టి వేర్వేరు size లో విడగొట్టడం** (IP waste తగ్గించడానికి). Approach: **పెద్ద అవసరం మొదట** allocate చెయ్యి.

```
Given: 192.168.1.0/24 (256 addresses). ఈ departments కి కావాలి:
  - Engineering: 100 hosts
  - Sales:       50 hosts
  - Support:     25 hosts
  - Point-to-point link: 2 hosts

పెద్దది మొదట (largest-first):
 ┌─────────────┬───────┬──────────┬──────────────────────┬─────────────┐
 │ Dept        │ కావాలి │ Prefix   │ Range                │ Broadcast   │
 ├─────────────┼───────┼──────────┼──────────────────────┼─────────────┤
 │ Engineering │ 100   │ /25 (126)│ .0   – .127          │ .127        │
 │ Sales       │ 50    │ /26 (62) │ .128 – .191          │ .191        │
 │ Support     │ 25    │ /27 (30) │ .192 – .223          │ .223        │
 │ P2P link    │ 2     │ /30 (2)  │ .224 – .227          │ .227        │
 └─────────────┴───────┴──────────┴──────────────────────┴─────────────┘
   మిగిలినది (.228–.255) → future use. IP waste minimal!
```

సూత్రం: ప్రతి అవసరానికి `2^h - 2 ≥ needed` కి సరిపడే smallest prefix ఎంచుకో, addresses ని వరుసగా allocate చెయ్యి. VLSM interview లో "efficiently subnet this network" అని అడిగినప్పుడు వాడతారు.

Supernetting / Route Aggregation

Subnetting యొక్క వ్యతిరేకం — supernetting. అనేక చిన్న networks ని ఒక్క పెద్ద route గా కలపడం. ఉదా: `192.168.0.0/24` + `192.168.1.0/24` → `192.168.0.0/23`. ఎందుకు? **Routing tables చిన్నగా ఉంచడానికి** (routers efficient). BGP (internet backbone) దీన్ని విస్తృతంగా వాడుతుంది — లేకపోతే routing tables కోట్ల entries అయిపోతాయి.

Routing Table — router ఎలా నిర్ణయిస్తుంది

Router లో ఒక **routing table** — "ఈ destination కి ఇలా వెళ్ళు":

```
Destination        Next Hop        Interface   Metric
0.0.0.0/0          49.207.0.1      eth0        1      ← default route (అన్నీ ఇక్కడికి)
192.168.1.0/24     0.0.0.0(local)  eth1        0      ← directly connected
10.0.0.0/8         192.168.1.254   eth1        5
```

- **Longest prefix match:** ఒక packet కి multiple routes match అయితే, **అత్యంత specific (longest prefix)** ని ఎంచుకుంటుంది. ఉదా 192.168.1.5 కి `/24` మరియు `/0` రెండూ match అయితే, `/24` (specific) గెలుస్తుంది.
- **Default route (0.0.0.0/0):** ఏ specific route match కాకపోతే ఇక్కడికి (నీ ISP/gateway).

MTU & Fragmentation (interview bonus)

MTU (Maximum Transmission Unit) = ఒక frame గరిష్ఠంగా ఎంత పెద్దది (bytes) కావచ్చు. Ethernet MTU = **1500 bytes** (standard). ఒక packet MTU కంటే పెద్దదైతే — **fragmentation** (ముక్కలు చేయడం) జరుగుతుంది, destination లో మళ్ళీ కలుపుతారు. IPv6 లో routers fragment చేయవు (sender చూసుకోవాలి — Path MTU Discovery). చాలా fragmentation = performance drop, అందుకే MTU tuning matters.

IPv6 — IPv4 అయిపోయాక

IPv4 4.3 billion addresses అయిపోయాయి. **IPv6 = 128 bits** → 2^{128} ≈ **340 undecillion** addresses (ప్రతి ఇసుక రేణువుకి కూడా ఒక IP ఇవ్వొచ్చు).

```
IPv4: 192.168.1.5                              (32 bits)
IPv6: 2001:0db8:85a3:0000:0000:8a2e:0370:7334  (128 bits)
      8 groups × 16 bits, hexadecimal, colon తో

Shorthand: 2001:db8:85a3::8a2e:370:7334
   (leading zeros తీసేయొచ్చు, వరుస zeros ని :: తో replace)
```

అంశం	IPv4	IPv6
Size	32 bits	128 bits
Addresses	~4.3 billion	~340 undecillion
రూపం	Decimal, dots (192.168.1.5)	Hex, colons (2001:db8::1)
NAT అవసరమా?	అవును (addresses తక్కువ)	అవసరం లేదు (సమృద్ధి)
Security	Optional (IPSec)	Built-in IPSec
Header	Complex, variable	Simplified, fixed

Routing — packet ఏ దారిలో వెళ్తుంది

Router = Layer 3 device. ఇది వేర్వేరు networks ని connect చేస్తుంది. ప్రతి packet కి **destination IP** చూసి, తన **routing table** ఆధారంగా "తర్వాత ఎక్కడికి పంపాలి (next hop)?" అని నిర్ణయిస్తుంది.

> **Analogy — GPS + highway signboards.** నువ్వు Hyderabad నుండి Delhi drive చేస్తున్నావు. ప్రతి junction లో ఒక signboard (router) "Delhi → ఎడమ వైపు" అని చెప్తుంది. ఆ signboard కి పూర్తి route తెలియదు — కేవలం "తర్వాతి మలుపు ఏది (next hop)" మాత్రమే తెలుసు. ప్రతి router ఇలా packet ని next hop కి పంపుతూ, hop-by-hop గా Delhi (destination) చేరుస్తుంది. **ఒక్క router కి మొత్తం internet route తెలియదు — తర్వాతి hop మాత్రమే.**

- **Static routing:** admin manually routes పెడతాడు (చిన్న networks).
- **Dynamic routing:** routers ఒకరితో ఒకరు మాట్లాడుకుని best route నేర్చుకుంటారు — protocols: **OSPF, BGP** (internet backbone BGP వాడుతుంది), RIP.
- **Default gateway:** నీ device కి తెలియని destination (వేరే network) కోసం packet ని పంపే router — సాధారణంగా నీ WiFi router (192.168.1.1).

NAT — Network Address Translation

సమస్య: నీ ఇంట్లో 5 devices (laptop, phone, TV...) ఉన్నాయి, కానీ నీ ISP ఒక్కటే **public IP** ఇచ్చింది. అయినా అన్నీ internet వాడగలవు — ఎలా? **NAT.**

NAT = private IPs ని ఒక public IP గా (మరియు తిరిగి) translate చేయడం. నీ router చేసేది ఇదే:

```
లోపల (private, LAN):              బయట (public, internet):
laptop  192.168.1.5  ┐
phone   192.168.1.6  ├──[Router NAT]──► ఒక్క public IP: 49.207.x.x
TV      192.168.1.7  ┘

Router ఒక table maintain చేస్తుంది:
192.168.1.5:52000  ↔  49.207.x.x:60001  (ఏ device ఏ request పంపిందో గుర్తు)
```

- Router ప్రతి outgoing request కి port number map చేసి, reply వచ్చినప్పుడు సరైన internal device కి తిరిగి పంపుతుంది (**PAT — Port Address Translation**).
- **ఎందుకు ముఖ్యం:** IPv4 addresses save అవుతాయి (billions of devices, limited IPs). అలాగే internal devices బయటికి కనిపించవు (security).
- **MERN relevance:** నీ local Node server (192.168.1.5:3000) internet నుండి direct గా access అవ్వదు (NAT వెనుక). అందుకే deploy చేయడానికి public server (AWS) లేదా ngrok లాంటి tunnel అవసరం.

ICMP & ping — network diagnostics

ICMP (Internet Control Message Protocol) = network errors, diagnostics కోసం. Data పంపదు, "control messages" పంపుతుంది (ఉదా "destination unreachable", "time exceeded").

`ping` ICMP వాడుతుంది: ఒక "echo request" పంపి, "echo reply" వస్తుందా, ఎంత time పడుతుందో (round-trip time) చూస్తుంది.

```
$ ping google.com
64 bytes from 142.250.x.x: icmp_seq=1 ttl=115 time=12.3 ms
                                                └ round-trip 12.3ms
```

- **ping = "నువ్వు బతికున్నావా, ఎంత దూరం?"** అని అడగడం. Server up ఉందా, latency ఎంత అని చూడటానికి.

DHCP — automatic IP assignment

సమస్య: నీ phone WiFi కి connect అయితే, దానికి IP address ఎవరిస్తారు? Manually పెట్టవు కదా. **DHCP (Dynamic Host Configuration Protocol)** automatic గా ఇస్తుంది.

DHCP DORA process (గుర్తుంచుకో — DORA):

```
Device:  "నాకు ఒక IP కావాలి!"          → DISCOVER (broadcast)
Server:  "ఇదిగో 192.168.1.5 తీసుకో"    → OFFER
Device:  "సరే, అదే కావాలి"              → REQUEST
Server:  "OK, అది నీది (24 గంటలు)"      → ACK (acknowledge)
```

- **DORA = Discover → Offer → Request → Acknowledge.**
- DHCP ఇచ్చేది: IP address, subnet mask, default gateway, DNS server addresses.
- **Lease:** IP permanent కాదు — కొంత time (lease) కి ఇస్తారు, తర్వాత renew చేయాలి. అందుకే నీ phone IP ఒక్కోసారి మారుతుంది.

Key Points

- **Network layer (L3) = routing + logical (IP) addressing.** Device = router. Data unit = packet.
- **IPv4 = 32 bits, 4 octets (0-255), ~4.3 billion.** IPv6 = 128 bits, దాదాపు unlimited.
- **Subnetting:** host bits = 32 − prefix; usable hosts = 2^(host bits) − 2 (network + broadcast తీసేయాలి). Block size = 256 − mask octet.
- **Private IPs:** 10.x, 172.16-31.x, 192.168.x (LAN, internet లో route అవ్వవు). 127.0.0.1 = localhost.
- **NAT = private IPs → ఒక public IP** (IPv4 save, security). నీ router చేస్తుంది.
- **Routing = hop-by-hop, router routing table + next hop చూస్తుంది.** BGP/OSPF dynamic routing.
- **ICMP/ping = diagnostics** (reachability, latency). **DHCP = automatic IP** (DORA: Discover-Offer-Request-Acknowledge).

Interview దృష్టి

- **"IPv4 vs IPv6?"** — 32 vs 128 bits, address exhaustion → IPv6. NAT IPv4 కి అవసరం, IPv6 కి కాదు. Format (dots/decimal vs colons/hex).
- **"Subnet 192.168.1.0/26 — ఎన్ని hosts, ranges?"** — host bits 6, 62 usable, 4 subnets, block size 64. Worked example 2 గుర్తుంచుకో. **Live గా calculate చేయగలగాలి.**
- **"What is NAT and why?"** — Private↔public IP translation. IPv4 addresses limited కాబట్టి multiple devices ఒక public IP share చేస్తాయి. Security bonus (internal hidden).
- **"How does a device get an IP?"** — DHCP, DORA process. Terminal knowledge: `ipconfig` / `ifconfig` .
- **"What is a default gateway?"** — వేరే network కి packets పంపే router (నీ WiFi router). తెలియని destination అన్నీ ఇక్కడికే.

- **"Difference between routing and switching?"** — Switch = L2, same network, MAC. Router = L3, వేర్వేరు networks, IP.
- **Gotcha:** "why usable hosts = 2^n − 2?" — network address (all-0 host) + broadcast address (all-1 host) reserved. /31 point-to-point links మాత్రం special (−2 apply కాదు, RFC 3021).
- **Gotcha:** ping fail అయితే server down అని కాదు — చాలా servers ICMP block చేస్తాయి (firewall). ping ≠ HTTP reachability.

---

6. Transport Layer — TCP, UDP, Handshakes, Flow/Congestion Control

వివరణ

Transport Layer (Layer 4) = end-to-end communication + reliability. Network layer (IP) packet ని destination *computer* కి చేరుస్తుంది. కానీ ఆ computer లో **ఏ application** కి? (browser? Node server? email?) — దాన్ని Transport layer **ports** ద్వారా చూస్తుంది. అలాగే data నమ్మకంగా (reliable), సరైన order లో చేరిందా అని ఇక్కడే చూస్తారు.

రెండు ప్రధాన protocols: **TCP** (reliable, ordered) మరియు **UDP** (fast, unreliable). నీ ప్రతి `fetch()` TCP వాడుతుంది. ఇది CN interview యొక్క **అత్యంత ముఖ్యమైన topic** — TCP handshake దాదాపు ప్రతి interview లో అడుగుతారు.

Real-life Scenario

> **Transport layer = ఒక భారీ పుస్తకం (book) ని courier చేసే dispatch manager.** నీ friend కి 500-పేజీల పుస్తకం పంపాలి. Network layer (postal system) ఒక్కో cover ని Delhi చేరుస్తుంది, కానీ మధ్యలో —
>
> - **పుస్తకం ఒకేసారి పట్టదు** → dispatch manager దాన్ని **పేజీలుగా (segments) విడగొట్టి**, ప్రతి పేజీకి **నంబరు (sequence number)** వేస్తాడు.
> - ఏ flat కి (**port**) చేరాలో cover మీద రాస్తాడు — building (IP) చాలదు, flat number కావాలి.
> - Friend ప్రతి పేజీ అందాక "పేజీ 1 అందింది, 2 పంపు (**acknowledgment**)" అని confirm పంపుతాడు.
> - ఏదైనా పేజీ పోతే → **మళ్ళీ పంపుతాడు (retransmission).** పేజీలు order తప్పి వస్తే → నంబర్లు చూసి **మళ్ళీ order లో పేర్చుతాడు.**
> - Friend నెమ్మదిగా చదివితే "ఆగు, ఇంకా 10 పేజీలే పంపు (**flow control**)" అంటాడు.
>
> **ఇదే TCP.** విడగొట్టడం, నంబరింగ్, confirmation, lost pages resend, ordering, speed control — అన్నీ ఒక్క reliable delivery కోసం. **UDP అయితే** — dispatch manager పేజీలు పంపేసి, confirmation అడగడు, పోతే పోయింది (fast కానీ risky). ఇప్పుడు detail గా.

Ports — ఏ application కి?

Port = ఒక computer లో ఏ application/service అని identify చేసే 16-bit number (0–65535).

> **Analogy — apartment building.** IP address = building address (ఏ building). Port = flat number (ఆ building లో ఏ flat). ఉత్తరం building కి వస్తే సరిపోదు — ఏ flat కి అని కూడా కావాలి. అలాగే packet computer కి వస్తే సరిపోదు — ఏ application (port) కి అని కావాలి.

Well-known ports (గుర్తుంచుకో):

Port	Protocol	వాడకం
80	HTTP	Web (unencrypted)
443	HTTPS	Web (encrypted) — నీ ప్రతి fetch
22	SSH	Secure remote login
53	DNS	Domain name resolution
25	SMTP	Email పంపడం
3306	MySQL	Database
27017	MongoDB	నీ MERN database!
3000	(dev)	నీ React/Node dev server

- **0–1023 = well-known ports** (standard services). **1024–49151 = registered. 49152–65535 = ephemeral** (client random ports).
- నీ browser `fetch()` చేసినప్పుడు: source port = random ephemeral (52000), dest port = 443 (HTTPS).

TCP vs UDP — deep comparison (interview must-know)

అంశం	TCP	UDP
పూర్తి పేరు	Transmission Control Protocol	User Datagram Protocol
Connection	Connection-oriented (handshake ముందు)	Connectionless (నేరుగా పంపేస్తుంది)
Reliability	Reliable (delivery guarantee, ack)	Unreliable (no guarantee)
Order	Ordered (sequence numbers)	Order guarantee లేదు
Speed	Slower (overhead ఎక్కువ)	Faster (overhead తక్కువ)
Error handling	Retransmit lost packets	పోతే పోయింది (no retransmit)
Flow/congestion control	ఉంది	లేదు
Header size	20–60 bytes	8 bytes (చిన్నది)
Data unit	Segment	Datagram
వాడకం	Web (HTTP), email, file transfer, DB	Video/audio streaming, gaming, DNS, VoIP
Analogy	**Phone call** (connect, confirm, hangup)	**Postcard** (పంపేసి మర్చిపో)

> **Analogy — TCP = registered post (with acknowledgment), UDP = ordinary postcard.**
>
> - **TCP:** నువ్వు registered post పంపుతావు. అవతలివారు receive చేసి **signature (ack)** ఇస్తారు. పోతే మళ్ళీ పంపుతారు. Slow కానీ guarantee. → banking, web pages (ఒక్క byte పోకూడదు).
> - **UDP:** postcard పంపేసి మర్చిపోతావు. చేరిందో లేదో తెలియదు, confirmation లేదు. Fast కానీ no guarantee. → live video (ఒక frame పోయినా పర్లేదు, next frame ముఖ్యం).

ఎప్పుడు ఏది? Data accuracy ముఖ్యమైతే TCP (web, file, DB — ఒక్క byte తప్పకూడదు). **Speed/real-time ముఖ్యమైతే, కొంత loss ఓకే అయితే UDP** (live streaming, gaming — పాత data కంటే తాజా data ముఖ్యం). ఆసక్తికరం: **HTTP/3 UDP వాడుతుంది** (QUIC protocol ద్వారా reliability జోడించి) — topic 8.

TCP 3-Way Handshake — connection establish (అత్యంత ముఖ్యం)

TCP data పంపేముందు ఒక **connection** establish చేస్తుంది — 3 steps లో. దీన్ని **3-way handshake** అంటారు. ప్రతి `fetch()` ముందు ఇది జరుగుతుంది.

```
Client (నీ browser)                          Server (GitHub)
   |                                            |
   |   ──────── 1. SYN ────────►                 |   "connect అవ్వాలి, నా seq=x"
   |         (SYN=1, seq=x)                     |
   |                                            |
   |   ◄─────── 2. SYN-ACK ────────             |   "సరే, నా seq=y, నీ x+1 అందింది"
   |        (SYN=1, ACK=1,                      |
   |         seq=y, ack=x+1)                    |
   |                                            |
```

```
      |   ──────── 3. ACK ──────────→      |   "నీ y అందింది, connection ready!"
      |        (ACK=1, ack=y+1)            |
      |                                    |
      |  ═════ CONNECTION ESTABLISHED ═════ |
      |   ──────── HTTP GET /users ────→   |   ఇప్పుడు actual data
```

అర్థం (SYN = Synchronize, ఒక flag):

1. **SYN:** Client → Server. "నేను connect అవ్వాలనుకుంటున్నాను, నా starting sequence number x."
2. **SYN-ACK:** Server → Client. "సరే (SYN), నీ x+1 అందింది (ACK). నా starting sequence number y."
3. **ACK:** Client → Server. "నీ y+1 అందింది (ACK). ఇప్పుడు మాట్లాడొచ్చు."

> **Analogy — phone call మొదలు.** (1) నువ్వు: "Hello?" (SYN). (2) అవతలివారు: "Hello, వినిపిస్తోందా?" (SYN-ACK). (3) నువ్వు: "అవును, వినిపిస్తోంది!" (ACK). ఇప్పుడు ఇద్దరూ ఒకరి మాట ఒకరికి వినిపిస్తోందని **confirm అయ్యింది** — అసలు మాట్లాడటం మొదలు. TCP కూడా ఇలాగే రెండువైపులా "send + receive పని చేస్తున్నాయి" అని confirm చేసుకుంటుంది.

ఎందుకు 3 steps (2 చాలవా)? రెండు వైపులా **send capability + receive capability** confirm అవ్వాలి. SYN (client→server) client పంపగలదని చూపిస్తుంది. SYN-ACK server receive+send చూపిస్తుంది. ACK client receive చూపిస్తుంది. **3 steps తో రెండు వైపులా bidirectional communication ready అని guarantee.**

TCP 4-Way Termination — connection close

Connection ముగించడానికి **4 steps** (ఎందుకు 4? రెండు వైపులా విడిగా close అవ్వాలి — full-duplex):

```
   Client                                  Server
      |   ──────── 1. FIN ──────────→     |   "నా పని అయ్యింది, close చేద్దాం"
      |   ←─────── 2. ACK ───────────     |   "సరే, నీ FIN అందింది"
      |          ...(server ఇంకా          |   (server మిగిలిన data పంపొచ్చు)
      |             data పంపొచ్చు)...     |
      |   ←─────── 3. FIN ───────────     |   "నా పని కూడా అయ్యింది"
      |   ──────── 4. ACK ──────────→     |   "సరే, close!"
      |        (TIME_WAIT: కొంతసేపు ఆగుతుంది)  |
```

1. **FIN:** Client → Server. "నా వైపు data అయిపోయింది."
2. **ACK:** Server → Client. "OK, గమనించాను."
3. **FIN:** Server → Client. "నా వైపు కూడా అయిపోయింది." (మధ్యలో server మిగిలిన data పంపొచ్చు — అందుకే 3+4 విడిగా.)
4. **ACK:** Client → Server. "OK, close." తర్వాత client **TIME_WAIT** state లో కొంతసేపు ఆగుతుంది (ఆలస్యంగా వచ్చే packets handle చేయడానికి).

ఎందుకు 4 (handshake 3, close 4)? Handshake లో SYN+ACK ని ఒకే packet లో కలపొచ్చు. Close లో server "నీ FIN అందింది (ACK)" వెంటనే చెప్పాలి, కానీ "నా data ఇంకా ఉంది" కాబట్టి తన FIN తర్వాత పంపుతుంది — అందుకే ACK, FIN విడిగా → 4 steps.

Sequence & Acknowledgment Numbers — reliability ఎలా

TCP data ని segments గా విడగొట్టి, ప్రతి byte కి **sequence number** ఇస్తుంది. Receiver "ఇంతవరకు అందింది, తర్వాతది ఇది పంపు" అని **acknowledgment number** పంపుతుంది.

```
Client పంపింది:  seq=100, data 50 bytes  (bytes 100–149)
Server reply:    ack=150  ("149 వరకు అందింది, 150 పంపు")
Client పంపింది:  seq=150, data 50 bytes  (bytes 150–199)
Server reply:    ack=200

ఒకవేళ packet పోతే:
Client:  seq=100 ✓, seq=150 ✗(పోయింది), seq=200 ✓
Server:  ack=150, ack=150, ack=150 ("ఇంకా 150 కావాలి!")
   → Client 150 ని retransmit చేస్తుంది (duplicate ACKs చూసి)
```

ఇదే TCP reliability: ప్రతి byte కి number, ప్రతి receipt కి ack. Ack రాకపోతే (timeout), లేదా duplicate acks వస్తే → **retransmit.** అందుకే TCP లో ఒక్క byte కూడా పోదు, order తప్పదు.

Flow Control — Sliding Window

సమస్య: Fast sender, slow receiver. Sender గబగబా పంపితే receiver buffer overflow అవుతుంది (data పోతుంది). **Flow control దీన్ని ఆపుతుంది.**

TCP Sliding Window: Receiver తన **buffer లో ఎంత ఖాళీ ఉందో** ("window size") ప్రతి ack లో చెప్తుంది. Sender ఆ window size కంటే ఎక్కువ పంపడు.

```
Receiver: "నా window = 4000 bytes (ఇంత మాత్రమే పంపు)"
Sender:   4000 bytes పంపి ఆగుతుంది, ack కోసం wait
Receiver: process చేసి "ఇప్పుడు window = 6000" (ఖాళీ పెరిగింది)
Sender:   ఇప్పుడు 6000 దాకా పంపొచ్చు
```

> **Analogy — నీళ్ళ ట్యాంక్ నింపడం.** అవతలివారి ట్యాంక్ (receiver buffer) చిన్నది. నువ్వు (sender) గొట్టం full గా తిప్పితే ట్యాంక్ పొంగి నీళ్ళు వృథా (packet loss). అవతలివారు "ఇంకా ఇంత ఖాళీ ఉంది (window)" అని చెప్తూ ఉంటే, నువ్వు ఆ మేరకే నింపుతావు. **Window = receiver ఎంత తీసుకోగలదో చెప్పే signal.**

"Sliding" ఎందుకు? Data ack అయ్యే కొద్దీ window ముందుకు "slide" అవుతుంది — కొత్త data పంపడానికి space వస్తుంది. ఇది receiver speed కి తగ్గట్టు adjust అవుతుంది.

Congestion Control — network రద్దీ

Flow control = receiver ని protect చేస్తుంది. Congestion control = network (routers between) ని protect చేస్తుంది. Network రద్దీగా (congested) ఉంటే packets drop అవుతాయి. TCP దీన్ని sense చేసి **పంపే rate తగ్గిస్తుంది.**

TCP congestion control phases (short):

1. **Slow Start:** మెల్లగా మొదలు, window ని exponential గా (1, 2, 4, 8...) పెంచుతూ.
2. **Congestion Avoidance:** ఒక threshold దాటాక, linear గా (నెమ్మదిగా) పెంచుతుంది.

3. **Packet loss detect అయితే:** window ని బాగా తగ్గించి (congestion signal), మళ్ళీ మొదలు.

> **Analogy — highway లో traffic sense చేయడం.** రోడ్డు ఖాళీగా ఉంటే వేగం పెంచుతావు (slow start). Traffic కనిపిస్తే మెల్లగా (avoidance). Jam అయితే బాగా slow అవుతావు (loss → reduce). TCP అందరూ కలిసి network ని క్రాష్ చేయకుండా, "fair share" వాడేలా చేస్తుంది.

Flow control vs Congestion control (interview): Flow control = **receiver** overflow ఆపడం (receiver window). Congestion control = **network** overload ఆపడం (congestion window). రెండూ TCP window ని control చేస్తాయి కానీ వేర్వేరు కారణాలకి. **Effective window = min(receiver window, congestion window)** — ఏది చిన్నదైతే అదే limit.

TCP Header — లోపల ఏముంది (interview bonus)

TCP segment header (20 bytes minimum) లో ఏ fields ఉంటాయో తెలిస్తే, TCP ఎలా పని చేస్తుందో గట్టిగా అర్థమవుతుంది:

```
 0                   16                   32 bits
┌──────────────────┬──────────────────┐
│ Source Port      │ Dest Port        │  ← ఏ app నుండి, ఏ app కి
├──────────────────┴──────────────────┤
│          Sequence Number            │  ← ఈ segment మొదటి byte number
├─────────────────────────────────────┤
│          Acknowledgment Number       │  ← "ఇంతవరకు అందింది, తర్వాతది పంపు"
├────┬────┬──────────┬─────────────────┤
│Data│Flag│  Window  │                 │  ← Flags: SYN/ACK/FIN/RST/PSH/URG
│Off │s   │  Size    │                 │    Window = flow control (buffer ఖాళీ)
├────┴────┴──────────┼─────────────────┤
│   Checksum         │  Urgent Ptr     │  ← error detection
├────────────────────┴─────────────────┤
│          Options (variable)           │
└───────────────────────────────────────┘
```

కీలక flags (control bits): SYN (connection start), **ACK** (acknowledgment), **FIN** (graceful close), **RST** (abrupt reset — "ఈ connection తప్పు, వెంటనే ఆపు"), **PSH** (వెంటనే deliver), **URG** (urgent data). Handshake లో SYN, close లో FIN — ఇవే ఆ flags. **UDP header (8 bytes) లో కేవలం 4 fields:** source port, dest port, length, checksum — అందుకే UDP fast (overhead తక్కువ).

TCP Connection States (netstat లో కనిపించేవి)

TCP connection ఒక **state machine** — ప్రతి connection ఒక state లో ఉంటుంది. `netstat` లో ఇవి కనిపిస్తాయి:

State	అర్థం
LISTEN	Server port తెరిచి connections కోసం wait (నీ Node server)
SYN_SENT	Client SYN పంపింది, reply కోసం wait
SYN_RECEIVED	Server SYN అందుకుని SYN-ACK పంపింది
ESTABLISHED	Connection ready, data flowing (active connections)
FIN_WAIT	Close మొదలుపెట్టింది (FIN పంపింది)
TIME_WAIT	Close అయ్యింది, late packets కోసం కొద్దిసేపు ఆగుతోంది
CLOSE_WAIT	అవతలివారు close చేశారు, మనం close చేయాలి

MERN gotcha: నీ server లో చాలా **CLOSE_WAIT** connections పేరుకుపోతే — నీ code connections properly close చేయట్లేదని అర్థం (resource leak). చాలా **TIME_WAIT** normal (busy server). `netstat -an | grep TIME_WAIT | wc -l` తో count చూడొచ్చు.

Congestion Control — లోతుగా (AIMD)

TCP congestion control యొక్క గుండె: **AIMD (Additive Increase, Multiplicative Decrease).**

- **Additive Increase:** అంతా బాగుంటే, congestion window ని నెమ్మదిగా (+1 ప్రతి RTT) పెంచు.
- **Multiplicative Decrease:** packet loss (congestion signal) అయితే, window ని **సగానికి తగ్గించు** (÷2).

```
window
   │        /\        /\          ← loss అయినప్పుడల్లా సగం (sawtooth pattern)
   │       /  \      /  \
   │      /    \    /    \
   │     /      \  /
   │____/        \/
   └──────────────────────────► time
    slow      loss → ÷2, మళ్ళీ నెమ్మదిగా పెరుగు
    start
```

- **Fast Retransmit:** 3 duplicate ACKs వస్తే (ఒకే byte మళ్ళీ మళ్ళీ అడుగుతున్నారు), timeout కోసం wait చేయకుండా వెంటనే retransmit.
- **Fast Recovery:** loss తర్వాత window ని 1 కి కాకుండా సగానికి తగ్గించి కొనసాగించడం (slow start మళ్ళీ మొదటి నుండి కాదు).
- **Variants:** TCP Reno, TCP Cubic (Linux default), BBR (Google — bandwidth-based, modern).

ఎందుకు ఇది తెలియాలి? SSE interview లో "నీ API కొన్నిసార్లు slow ఎందుకు?" అంటే — network congestion → TCP throughput తగ్గింది అని reason చెప్పగలగాలి. అలాగే "bufferbloat", "why does a single packet loss hurt throughput?" లాంటివి.

Key Points

- **Transport layer (L4) = end-to-end + ports (ఏ app) + reliability.** TCP & UDP.

- **Port = 16-bit app identifier.** 80=HTTP, 443=HTTPS, 53=DNS, 22=SSH, 27017=MongoDB.
- **TCP = connection-oriented, reliable, ordered, slower** (web, email, DB). **UDP = connectionless, unreliable, fast** (video, gaming, DNS).
- **3-way handshake (SYN → SYN-ACK → ACK)** — connection establish. రెండు వైపులా send+receive confirm.
- **4-way termination (FIN → ACK → FIN → ACK)** — close. TIME_WAIT చివర్లో.
- **Sequence + ack numbers = reliability** (ప్రతి byte number, lost → retransmit).
- **Flow control (sliding window) = receiver protect** (receiver buffer). **Congestion control = network protect** (congestion window, slow start).

Interview దృష్టి

- **"Explain TCP 3-way handshake."** — SYN, SYN-ACK, ACK. Sequence numbers. **Diagram గీయగలగాలి.** ఎందుకు 3 (bidirectional confirm) చెప్పు. ఇది almost guaranteed question.
- **"TCP vs UDP?"** — Table నుండి 4-5 points (connection, reliability, order, speed, use cases). "TCP = phone call, UDP = postcard."
- **"Why does connection close take 4 steps but open takes 3?"** — Open: SYN+ACK combine. Close: full-duplex, ప్రతి direction విడిగా close (server కి ఇంకా data ఉండొచ్చు) → FIN, ACK విడిగా.
- **"Flow control vs congestion control?"** — Flow = receiver overload (receiver window). Congestion = network overload (congestion window, slow start). రెండూ TCP.
- **"How does TCP ensure reliability?"** — Sequence numbers + acknowledgments + retransmission (timeout/duplicate ACK) + checksums + ordering.
- **"What is TIME_WAIT?"** — Client closing తర్వాత కొంతసేపు (2×MSL) ఆగుతుంది — late/duplicate packets clean గా handle చేయడానికి, పాత connection తో కొత్త connection confuse అవ్వకుండా.
- **Gotcha:** "TCP fast or slow?" — Slow*er* than UDP (handshake + acks overhead) కానీ reliable. Trade-off: reliability vs speed.
- **Gotcha:** DNS ఎందుకు UDP? — చిన్న request/reply, speed ముఖ్యం, retry easy. కానీ పెద్ద responses (zone transfer) TCP వాడతాయి. HTTP/3 UDP (QUIC) వాడటం modern trend.

---

7. Application Layer — Overview (HTTP, DNS, SMTP, FTP, WebSockets)

వివరణ

Application Layer (Layer 7) = నువ్వు (developer) నేరుగా పని చేసే layer. ఇక్కడ మనుషులకి/apps కి అర్థమయ్యే **protocols** ఉంటాయి — HTTP (web), DNS (names), SMTP (email), FTP (files), WebSocket (realtime). ఈ protocols అన్నీ కింద ఉన్న Transport layer (TCP/UDP) ని వాడతాయి, కానీ నీకు ఆ వివరాలు కనిపించవు.

ముఖ్యమైన స్పష్టత: Application layer అంటే **నీ React app కాదు** — నీ app వాడే protocols. నువ్వు `fetch()` రాసినప్పుడు, browser ఒక **HTTP** (application layer protocol) message తయారు చేస్తుంది. ఈ topic ఒక **overview** —

HTTP (topic 8) మరియు DNS (topic 10) ని తర్వాత లోతుగా చూద్దాం. ఇక్కడ అన్ని application protocols ఒక్కచోట map చేసుకుందాం.

Real-life Scenario

> **Application layer = వేర్వేరు పనులకి వేర్వేరు ప్రత్యేక సేవలు (specialized services).** ఒక పెద్ద మాల్ ఊహించు:
>
> - **HTTP** = information desk (ఏదైనా అడిగితే page ఇస్తుంది — web browsing).
> - **DNS** = directory/enquiry (పేరు చెప్తే location చెప్తుంది — "GitHub ఎక్కడ?" → IP).
> - **SMTP** = post office (ఉత్తరాలు/emails పంపడం).
> - **FTP** = cargo/courier counter (పెద్ద files పంపడం).
> - **WebSocket** = intercom line (రెండువైపులా continuous మాట్లాడటం — live chat).
>
> ప్రతి service ఒక specific పని చేస్తుంది, తనదైన "భాష" (protocol) మాట్లాడుతుంది. కానీ అన్నీ కింది రవాణా (TCP/UDP) మీదే ఆధారపడతాయి. **నీ MERN app రోజూ HTTP, DNS, WebSocket వాడుతుంది.**

ప్రధాన Application Layer Protocols

```
Application Layer Protocols
├── HTTP/HTTPS   (80/443)  → web pages, REST APIs   [TCP]  ← నీ fetch()
├── DNS          (53)      → domain name → IP        [UDP]  ← ప్రతి URL ముందు
├── SMTP         (25/587)  → email పంపడం             [TCP]
├── IMAP/POP3    (143/110) → email చదవడం              [TCP]
├── FTP          (20/21)   → file transfer           [TCP]
├── WebSocket    (80/443)  → realtime bidirectional  [TCP]  ← live chat/notifications
├── SSH          (22)      → secure remote shell      [TCP]
└── DHCP         (67/68)   → automatic IP             [UDP]
```

HTTP — HyperText Transfer Protocol (topic 8 లో deep)

- **పని:** Web యొక్క పునాది. Client (browser) request పంపుతుంది, server response ఇస్తుంది. నీ ప్రతి `fetch()` , ప్రతి REST API call HTTP.
- **Stateless:** ప్రతి request స్వతంత్రం — server గత request గుర్తుంచుకోదు (cookies/tokens తో state manage చేస్తారు).
- **TCP మీద** (port 80), HTTPS = HTTP + TLS encryption (port 443).
- **MERN:** ఇది నీ రోజువారీ protocol. Topic 8 లో methods, status codes, headers అన్నీ deep గా.

DNS — Domain Name System (topic 10 లో deep)

- **పని: Human-friendly names ని IP addresses గా మార్చడం.** `github.com` → `140.82.112.3` . Computers IP తో మాట్లాడతాయి, మనుషులు names గుర్తుంచుకుంటారు — DNS ఆ bridge.
- **"Internet యొక్క phonebook."** ప్రతి URL type చేసినప్పుడు మొదట DNS lookup జరుగుతుంది.
- **UDP మీద** (port 53, fast). Topic 10, 11 లో deep.

SMTP — Simple Mail Transfer Protocol

- **పని: Email పంపడం** (sending). నీ email client → mail server → recipient mail server.
- **గమనిక:** SMTP **పంపడానికి** మాత్రమే. **చదవడానికి** IMAP (port 143) లేదా POP3 (port 110) వాడతారు.
- **Analogy:** SMTP = post office లో ఉత్తరం **పోస్ట్ చేయడం**. IMAP/POP3 = నీ ఇంటి **letterbox నుండి తీసుకోవడం**.
- **MERN:** నీ Node app "signup email" పంపేటప్పుడు (Nodemailer, SendGrid) కింద SMTP వాడుతుంది.

FTP — File Transfer Protocol

- **పని:** Computers మధ్య **files transfer** చేయడం (upload/download).
- **విశేషం:** రెండు connections వాడుతుంది — **port 21 (control**, commands కి) + **port 20 (data**, actual file కి).
- **Security:** పాత FTP unencrypted (insecure). ఇప్పుడు **SFTP** (SSH మీద) లేదా **FTPS** (TLS మీద) వాడతారు.
- **గమనిక:** ఇప్పుడు చాలావరకు files ని HTTP/HTTPS (cloud storage, S3) ద్వారానే transfer చేస్తారు — FTP legacy అవుతోంది.

WebSocket — realtime bidirectional (MERN కి ముఖ్యం)

సమస్య: HTTP **request-response** మాత్రమే — client అడిగితేనే server జవాబిస్తుంది. Server తనంతట తానుగా client కి push చేయలేదు. కానీ live chat, notifications, live scores కి server → client push కావాలి. **WebSocket దీన్ని solve చేస్తుంది.**

```
HTTP (traditional):                WebSocket (persistent):
Client → request → Server          Client ↔↔↔↔↔ Server
Client ← response ← Server           (ఒకసారి connect, తర్వాత
(ప్రతిసారి కొత్త connection)         రెండువైపులా ఎప్పుడైనా push)
```

- **Persistent, full-duplex connection:** ఒకసారి establish అయ్యాక, రెండు వైపులా ఎప్పుడైనా data పంపొచ్చు (server కూడా push చేయొచ్చు).
- **ఎలా మొదలవుతుంది:** ఒక HTTP request తో మొదలై, "Upgrade: websocket" header తో connection ని WebSocket కి **upgrade** చేస్తారు (handshake). తర్వాత అదే TCP connection మీద continuous.
- **MERN: Socket.IO** (Node) — live chat, notifications, collaborative editing, live dashboards. నీ realtime features అన్నీ ఇది.

HTTP vs WebSocket (interview)

అంశం	HTTP	WebSocket
Communication	Request-response (one-way trigger)	Full-duplex (రెండువైపులా)
Connection	ప్రతి request కి కొత్తది (keep-alive తప్ప)	Persistent (ఒకసారి)
Server push?	కాదు (client అడగాలి)	అవును (ఎప్పుడైనా)
Overhead	ప్రతి request కి headers	Handshake తర్వాత తక్కువ
వాడకం	REST APIs, web pages	Chat, live updates, gaming
Protocol	http:// / https://	ws:// / wss://

గుర్తుంచుకో: WebSocket కి ముందు, server push simulate చేయడానికి **polling** లేదా **long polling** వాడేవారు. WebSocket వీటన్నిటిని replace చేసింది.

Realtime techniques comparison (MERN interview favorite)

"Live feature ఎలా build చేస్తావు?" అంటే ఈ 4 options తెలియాలి:

Technique	ఎలా పని చేస్తుంది	Direction	ఖర్చు	ఎప్పుడు
Short Polling	ప్రతి X sec కి కొత్త request	Client pulls	ఎక్కువ (waste)	Simple, rare updates
Long Polling	Request ని open గా ఉంచి, data వచ్చినప్పుడు reply	Client pulls (delayed)	మధ్యస్థం	WebSocket support లేని చోట
SSE (Server-Sent Events)	ఒక HTTP connection మీద server continuously push	Server → client (one-way)	తక్కువ	Notifications, live feed, stock prices
WebSocket	Persistent full-duplex connection	రెండువైపులా	తక్కువ	Chat, gaming, collaboration

```
Short Polling:   C→S? C←S(no). C→S? C←S(no). C→S? C←S(data!)   ← waste
Long Polling:    C→S? .....(server holds)..... C←S(data!)        ← better
SSE:             C→S(connect once), then S→→→→→C (stream)        ← one-way push
WebSocket:       C⟷S (persistent), రెండువైపులా ఎప్పుడైనా          ← best bidirectional
```

SSE vs WebSocket (interview trap): SSE = server→client one-way (HTTP మీద, auto-reconnect, simpler, text only). **WebSocket = bidirectional** (binary+text, కానీ setup complex). **Notifications/live feed కి SSE సరిపోతుంది; chat/gaming కి WebSocket కావాలి.** చాలామంది WebSocket ని అన్నిటికీ వాడతారు — కానీ one-way అయితే SSE efficient.

WebSocket lifecycle (Socket.IO వెనుక)

```
1. HTTP request:  GET /chat  +  "Upgrade: websocket" header
2. Server reply:  101 Switching Protocols  ← handshake success
3. తర్వాత: అదే TCP connection మీద frames (ws:// / wss://)
   - client.send()  →  server
   - server.emit()  →  client   (ఎప్పుడైనా, రెండువైపులా)
4. Ping/pong frames తో connection alive check (heartbeat)
5. close() → connection ముగింపు
```

- **wss://** = secure WebSocket (TLS మీద, HTTPS లాంటిది). ప్రతి production app లో wss.
- **MERN:** Socket.IO ఇదంతా abstract చేస్తుంది + fallback (WebSocket support లేకపోతే long polling), rooms, reconnection, broadcast. నీ live chat, notifications, live dashboards దీని మీద.
- **Scaling gotcha:** WebSocket persistent connections stateful — multiple servers వాడితే, ఒక user ఏ server కి connect అయ్యాడో track చేయాలి (sticky sessions లేదా Redis pub/sub adapter). Stateless HTTP కంటే scale కష్టం.

Key Points

- **Application layer (L7) = నువ్వు వాడే protocols** (నీ app కాదు). HTTP, DNS, SMTP, FTP, WebSocket.
- **HTTP (80/443, TCP)** = web/REST, stateless, request-response. నీ fetch().
- **DNS (53, UDP)** = name → IP, "internet phonebook".
- **SMTP (25, TCP)** = email పంపడం; IMAP/POP3 = చదవడం.
- **FTP (20/21, TCP)** = file transfer (control + data connections); ఇప్పుడు SFTP/HTTPS.
- **WebSocket (ws://, wss://)** = persistent full-duplex, server push సాధ్యం; live chat/notifications (Socket.IO). HTTP upgrade తో మొదలు.

Interview దృష్టి

- **"HTTP vs WebSocket?"** — HTTP request-response (server push కాదు), WebSocket persistent full-duplex (server push సాధ్యం). Chat/live = WebSocket. (Table remember.)
- **"How does WebSocket start?"** — HTTP handshake తో మొదలు, "Upgrade: websocket" header, తర్వాత అదే connection persistent full-duplex.
- **"Which protocol for email? Send vs receive?"** — Send = SMTP. Receive = IMAP (server-side, sync) / POP3 (download+delete).
- **"Which transport does DNS use, and why?"** — UDP (fast, small query). Large responses → TCP.
- **"How would you build a live notification feature in MERN?"** — WebSocket (Socket.IO). Alternatives: SSE (Server-Sent Events, one-way server→client), long polling (fallback).
- **Gotcha:** SSE vs WebSocket — SSE = server→client one-way only (over HTTP, simpler, auto-reconnect). WebSocket = bidirectional. Notifications కి SSE సరిపోతుంది, chat కి WebSocket.

Part 3 — Web & Practical (నీ MERN ప్రపంచం)

> ఇప్పటివరకు theory. ఇప్పుడు **నీ రోజువారీ ప్రపంచం** — HTTP, HTTPS/TLS, DNS, మరియు "URL enter చేస్తే ఏం జరుగుతుంది". ఈ Part నీకు అత్యంత relevant — ఎందుకంటే నువ్వు ఇవి రోజూ వాడతావు (fetch, cookies, HTTPS), కానీ లోపల ఎలా పని చేస్తాయో ఇప్పుడు గట్టిగా తెలుస్తుంది. SSE interview లో ఈ Part నుండే ఎక్కువ ప్రశ్నలు వస్తాయి — నీ MERN అనుభవాన్ని theory తో connect చేసే చోటు ఇది.

---

8. HTTP Deep — Methods, Status Codes, Headers, Cookies, HTTP/1.1 vs 2 vs 3

వివరణ

HTTP (HyperText Transfer Protocol) = web మొత్తం నడిచే protocol. నీ browser ↔ server మధ్య ప్రతి communication HTTP. నువ్వు రోజూ `fetch()` , `axios` , Express routes రాస్తావు — ఇవన్నీ HTTP. ఇప్పుడు లోపల ఏముందో పూర్తిగా చూద్దాం.

HTTP యొక్క 2 కీలక లక్షణాలు:

1. **Request-Response model:** Client అడుగుతుంది (request), server జవాబిస్తుంది (response). Server తనంతట తానుగా మాట్లాడదు.
2. **Stateless:** ప్రతి request స్వతంత్రం. Server గత request ని గుర్తుంచుకోదు. (State కావాలంటే cookies/tokens వాడతాం — కింద.)

Real-life Scenario

> **HTTP = restaurant లో waiter తో మాట్లాడటం.**
>
> - నువ్వు (client) waiter (server) ని పిలిచి **"ఒక masala dosa" (request)** అంటావు. Waiter **dosa తెచ్చి (response)** ఇస్తాడు.
> - ప్రతిసారి **నువ్వే పిలవాలి** — waiter తనంతట తానుగా రాడు (request-response).
> - Waiter నిన్ను **గుర్తుంచుకోడు** — తర్వాత "ఇంకో dosa" అంటే, "మీరు ఎవరు, ముందు ఏం order చేశారు?" అని మళ్ళీ మొదటి నుండి (stateless). నిన్ను గుర్తుపెట్టుకోవాలంటే నీకు ఒక **token/bill number (cookie)** ఇస్తాడు — ప్రతిసారి అది చూపిస్తే గుర్తుపడతాడు.
>
> ఇదే HTTP. Request-response + stateless. State కావాలంటే cookie అనే "token" వాడతాం.

HTTP Request Structure

ఒక HTTP request 4 భాగాలు:

```
┌──────────────────────────────────────────────────────┐
│ GET /api/users/42 HTTP/1.1          ← 1. Request line │
│                                     (method, path, version)
│ Host: api.github.com                ← 2. Headers      │
│ Authorization: Bearer xyz123                          │
│ Accept: application/json                              │
│ User-Agent: Mozilla/5.0                               │
│                                     ← 3. ఖాళీ line     │
│ { "name": "John" }                  ← 4. Body (POST/PUT)│
└──────────────────────────────────────────────────────┘
```

1. **Request line:** method (GET) + path (/api/users/42) + HTTP version.
2. **Headers:** metadata (key: value) — Host, Authorization, Accept, Content-Type...
3. **Empty line:** headers ముగిశాయని గుర్తు.
4. **Body:** actual data (POST/PUT లో — JSON payload). GET లో సాధారణంగా body ఉండదు.

HTTP Response Structure

```
┌──────────────────────────────────────────────────────┐
│ HTTP/1.1 200 OK                     ← 1. Status line  │
│                                     (version, code, text)
│ Content-Type: application/json      ← 2. Headers      │
│ Content-Length: 128                                   │
│ Set-Cookie: session=abc123                            │
│                                     ← 3. ఖాళీ line     │
│ { "id": 42, "name": "John" }        ← 4. Body         │
└──────────────────────────────────────────────────────┘
```

HTTP Methods (verbs) — ఏం చేయాలి

Method	పని	Idempotent?	Safe?	MERN ఉదాహరణ
GET	Data చదవడం (read)	అవును	అవును	users list పొందడం
POST	కొత్తది సృష్టించడం (create)	**కాదు**	కాదు	కొత్త user signup
PUT	పూర్తిగా update/replace	అవును	కాదు	user profile మొత్తం మార్చడం
PATCH	పాక్షికంగా update	కాదు*	కాదు	user email మాత్రం మార్చడం
DELETE	తొలగించడం	అవును	కాదు	user account delete
HEAD	GET లాంటిదే కానీ body లేదు (headers only)	అవును	అవును	file ఉందా, size ఎంత
OPTIONS	ఏ methods allowed	అవును	అవును	**CORS preflight**

- **Safe = server data ని మార్చదు** (GET, HEAD). **Idempotent = ఎన్నిసార్లు చేసినా ఫలితం ఒకటే** (GET, PUT, DELETE). POST idempotent కాదు — 3 సార్లు POST → 3 users create అవుతాయి.
- **MERN gotcha — CORS preflight:** నీ React app (localhost:3000) వేరే origin (api.com) కి POST చేస్తే, browser ముందు ఒక **OPTIONS** request (preflight) పంపి "ఈ origin కి allow ఉందా?" అని అడుగుతుంది. Server సరైన

CORS headers (`Access-Control-Allow-Origin`) ఇవ్వాలి. ఇది చాలామంది MERN devs కి తెలియని కారణంగా "CORS error" వస్తుంది.

HTTP Status Codes — server ఏం చెప్తోంది

5 categories (మొదటి digit ముఖ్యం):

```
1xx  → Informational (అరుదు)      "పని జరుగుతోంది"
2xx  → Success                    "అయ్యింది! ✓"
3xx  → Redirection                "వేరే చోటికి వెళ్ళు"
4xx  → Client Error (నీ తప్పు)     "నువ్వు తప్పు అడిగావు"
5xx  → Server Error (server తప్పు) "నా (server) దగ్గర సమస్య"
```

Code	అర్థం	ఎప్పుడు (MERN)
200 OK	Success	GET/PUT సఫలం
201 Created	కొత్తది సృష్టించబడింది	POST success (కొత్త user)
204 No Content	Success కానీ body లేదు	DELETE success
301 Moved Permanently	శాశ్వతంగా మారింది	old URL → new URL (SEO)
302 Found	తాత్కాలిక redirect	login తర్వాత redirect
304 Not Modified	Cache లోనిది వాడు	conditional GET (caching)
400 Bad Request	Request తప్పు	invalid JSON, missing fields
401 Unauthorized	Login కాలేదు	token లేదు/invalid
403 Forbidden	Login అయింది కానీ అనుమతి లేదు	user admin route access
404 Not Found	వనరు లేదు	తప్పు URL
409 Conflict	ఘర్షణ	duplicate email signup
422 Unprocessable	Validation fail	form validation errors
429 Too Many Requests	Rate limit	API throttling
500 Internal Server Error	Server crash	unhandled exception (నీ Node code)
502 Bad Gateway	Upstream server bad	proxy/load balancer వెనుక server down
503 Service Unavailable	Server busy/down	maintenance, overload
504 Gateway Timeout	Upstream timeout	backend నెమ్మది

401 vs 403 (favorite interview trap): 401 = "నువ్వు ఎవరో తెలియదు" (authentication fail — login చెయ్యి). 403 = "నువ్వు ఎవరో తెలుసు కానీ నీకు అనుమతి లేదు" (authorization fail — admin కాదు).

HTTP Headers — ముఖ్యమైనవి

Header	పని	MERN
Content-Type	Body ఏ format	`application/json` , `text/html`
Authorization	Auth token	`Bearer <JWT>`
Accept	Client ఏ format కావాలి	`application/json`
Cookie	Client → server cookies	session ID
Set-Cookie	Server → client cookie set	login తర్వాత
Cache-Control	Caching rules	`max-age=3600` , `no-cache`
Content-Length	Body size (bytes)	—
User-Agent	Client ఏ browser/app	—
Access-Control-Allow-Origin	CORS	`*` లేదా specific origin
ETag	Version identifier (caching)	conditional requests

Cookies vs Sessions vs Tokens — state ఎలా maintain

HTTP stateless. కానీ "user logged in" అని గుర్తుంచుకోవాలి. 2 approaches:

```
—— Session-based (server గుర్తుంచుకుంటుంది) ——
1. Login → server session create చేసి, DB/memory లో store
2. Server "Set-Cookie: sessionId=abc" పంపుతుంది
3. Browser ప్రతి request కి "Cookie: sessionId=abc" జోడిస్తుంది
4. Server abc ని lookup చేసి "ఇది John" అని తెలుసుకుంటుంది
   → State SERVER లో. Cookie కేవలం ID.

—— Token-based / JWT (stateless) ——
1. Login → server ఒక signed JWT (encoded user info) ఇస్తుంది
2. Client దాన్ని store (localStorage/cookie)
3. ప్రతి request కి "Authorization: Bearer <JWT>"
4. Server JWT signature verify చేస్తుంది (DB lookup అవసరం లేదు)
   → State CLIENT లో (token లోనే). Server stateless.
```

అంశం	Session (cookie)	JWT (token)
State ఎక్కడ	Server (DB/memory)	Client (token లోనే)
Scaling	కష్టం (session store share)	సులభం (stateless)
Revoke	సులభం (server delete)	కష్టం (expiry దాకా valid)
Size	చిన్న ID	పెద్ద token (ప్రతి request)

- **Cookie attributes (security — interview):** `HttpOnly` (JS access చేయలేదు — XSS నుండి రక్ష), `Secure` (HTTPS లో మాత్రమే), `SameSite` (CSRF నుండి రక్ష — Strict/Lax/None).

Keep-Alive & HTTP versions

Keep-Alive (persistent connection): HTTP/1.0 లో ప్రతి request కి కొత్త TCP connection (handshake ప్రతిసారి — slow). HTTP/1.1 లో **keep-alive** — ఒకే TCP connection ని multiple requests కి reuse చేస్తారు. `Connection: keep-alive` header.

HTTP versions పరిణామం:

Version	ముఖ్య మెరుగుదల	సమస్య
HTTP/1.0	Basic, ప్రతి request కి కొత్త connection	Slow (handshake ప్రతిసారి)
HTTP/1.1	Keep-alive, pipelining, Host header	**Head-of-line blocking** (ఒక slow request మిగతావాటిని ఆపుతుంది)
HTTP/2	**Multiplexing** (ఒక connection లో parallel), header compression, server push, binary	ఒక TCP connection → **TCP-level HOL blocking** (packet loss అందరినీ ఆపుతుంది)
HTTP/3	**QUIC (UDP మీద)**, TCP HOL blocking పోయింది, faster handshake, built-in TLS	కొత్తది, adoption పెరుగుతోంది

కీలక పరిణామం అర్థం:

- **HTTP/1.1 సమస్య:** ఒకే connection లో requests వరుసగా — ఒక slow response మిగతావాటిని block చేస్తుంది (head-of-line blocking). Browsers దీన్ని దాటడానికి 6 parallel connections తెరిచేవి.
- **HTTP/2 పరిష్కారం: Multiplexing** — ఒకే connection లో అనేక requests/responses **ఏకకాలంలో (interleaved)**. కానీ ఇంకా TCP మీదే → ఒక TCP packet పోతే అన్ని streams ఆగుతాయి (TCP-level HOL).
- **HTTP/3 పరిష్కారం: UDP-based QUIC** — TCP ని వదిలేసి, ప్రతి stream స్వతంత్రం (ఒకటి loss అయినా మిగతావి కొనసాగుతాయి). Handshake కూడా faster (TLS + connection ఒకేసారి).

REST — API design style

REST (Representational State Transfer) = HTTP ని వాడి APIs design చేసే ఒక style/convention. నీ Express routes ఇవే.

- **Resources ని URLs గా:** `/users` , `/users/42` , `/users/42/posts` .
- **Methods ని actions గా:** GET (read), POST (create), PUT/PATCH (update), DELETE (delete).
- **Stateless:** ప్రతి request self-contained (auth token ప్రతిసారి).

```
GET    /users        → అందరు users
GET    /users/42     → user 42
POST   /users        → కొత్త user సృష్టి
```

```
PUT    /users/42   → user 42 update
DELETE /users/42   → user 42 delete
```

REST principles (interview): stateless, client-server separation, uniform interface (resources+methods), cacheable, layered.

REST vs GraphQL vs gRPC (modern API styles — SSE లో అడుగుతారు):

అంశం	REST	GraphQL	gRPC
Data fetch	Fixed endpoints	Client ఏ fields కావాలో అడుగుతుంది	RPC methods (function call లా)
Over/under-fetching	సమస్య (fixed shape)	పరిష్కారం (exact fields)	తక్కువ (typed)
Format	JSON (text)	JSON	Protocol Buffers (binary)
Transport	HTTP/1.1, /2	HTTP (POST)	**HTTP/2**
Speed	మధ్యస్థం	మధ్యస్థం	**అత్యంత fast** (binary)
వాడకం	Public APIs, general web	Complex data (mobile, dashboards)	Microservices (internal)
MERN	Express routes (common)	Apollo Server	Node microservices

- **Over-fetching:** REST లో `/users/42` పంపితే అన్ని fields వస్తాయి (నీకు name మాత్రమే కావాలన్నా). **Under-fetching:** ఒక screen కి multiple endpoints call చేయాలి. GraphQL ఈ రెండింటినీ solve చేస్తుంది — ఒక్క query లో exact data.
- **ఎప్పుడు ఏది:** Public/simple API = REST. Mobile/complex nested data = GraphQL. Internal high-performance microservices = gRPC.

Real curl session — HTTP ని చేతితో చూడటం:

```
# GET request, headers సహా చూడటం (-v = verbose)
$ curl -v https://api.github.com/users/torvalds
> GET /users/torvalds HTTP/2       ← request line
> Host: api.github.com             ← request headers
> User-Agent: curl/8.0
< HTTP/2 200                        ← response status
< content-type: application/json    ← response headers
< { "login": "torvalds", ... }      ← body

# POST request (JSON body పంపడం)
$ curl -X POST https://api.example.com/users \
       -H "Content-Type: application/json" \
       -H "Authorization: Bearer TOKEN" \
       -d '{"name": "John"}'

# Headers మాత్రమే (HEAD-like)
```

```
$ curl -I https://github.com
HTTP/2 200
```

ఇది నీ browser DevTools → Network tab చూపించేదే — కానీ terminal లో. Interview లో "API debug ఎలా?" అంటే curl mention చెయ్యి.

Key Points

- **HTTP = request-response + stateless.** ప్రతి request స్వతంత్రం.
- **Request = request line + headers + body. Response = status line + headers + body.**
- **Methods:** GET (read, safe, idempotent), POST (create, not idempotent), PUT (replace), PATCH (partial), DELETE. OPTIONS = CORS preflight.
- **Status codes:** 2xx success, 3xx redirect, 4xx client error, 5xx server error. **401 = auth fail, 403 = permission fail.**
- **State:** Session (cookie ID, server state) vs JWT (token, client state, stateless, scalable).
- **Cookie security:** HttpOnly (XSS), Secure (HTTPS), SameSite (CSRF).
- **HTTP/1.1 (keep-alive, HOL blocking) → HTTP/2 (multiplexing) → HTTP/3 (QUIC/UDP, no TCP HOL).**
- **REST = resources(URLs) + methods(verbs) + stateless.**

Interview దృష్టి

- **"HTTP methods తేడా? Idempotency అంటే?"** — GET/PUT/DELETE idempotent (ఎన్నిసార్లైనా same result), POST కాదు. Safe = GET/HEAD (data మార్చదు).
- **"401 vs 403?"** — 401 = not authenticated (login చెయ్యి). 403 = authenticated కానీ not authorized (అనుమతి లేదు).
- **"HTTP/1.1 vs 2 vs 3?"** — 1.1 keep-alive కానీ HOL blocking; 2 multiplexing (parallel over one connection) కానీ TCP HOL; 3 QUIC/UDP (no TCP HOL, faster). పరిణామం చెప్పు.
- **"Cookies vs JWT/sessions?"** — Session = server state (revoke easy, scale hard). JWT = client state, stateless (scale easy, revoke hard). Trade-offs.
- **"What is CORS?"** — Browser security: వేరే origin కి request చేస్తే server అనుమతి (Access-Control-Allow-Origin) కావాలి. Non-simple requests → OPTIONS preflight ముందు.
- **"Is HTTP stateless? Then how does login work?"** — HTTP stateless. Login state = cookies (session ID) లేదా tokens (JWT) ద్వారా. ప్రతి request తో identity పంపుతాం.
- **Gotcha:** "GET can have a body?" — Technically అవును కానీ practice లో ignore చేస్తారు. GET body వాడొద్దు.
- **Gotcha:** HTTP/2 server push deprecated అవుతోంది (browsers support తగ్గించారు) — బదులు `preload` hints. HTTP/3 ముఖ్యం.

---

9. HTTPS & TLS — Encryption, Handshake, Certificates

వివరణ

HTTPS = HTTP + TLS (encryption). నీ `fetch("https://...")` లో ఆ `s` అంటే "secure" — TLS ద్వారా encrypted. HTTP plain text లో పంపుతుంది (ఎవరైనా చదవొచ్చు); HTTPS దాన్ని encrypt చేసి, tamper చేయకుండా, సరైన server తోనే మాట్లాడుతున్నామని నిర్ధారిస్తుంది.

HTTPS ఇచ్చే 3 guarantees (CIA లో):

1. **Confidentiality (గోప్యత):** Data encrypted — మధ్యలో ఎవరూ (hacker, ISP) చదవలేరు.
2. **Integrity (సమగ్రత):** Data మధ్యలో మార్చబడలేదని నిర్ధారణ (tampering detection).
3. **Authentication (ప్రామాణికత):** నువ్వు మాట్లాడేది నిజంగా `github.com` తోనే, ఒక fake server తో కాదు (certificate ద్వారా).

TLS (Transport Layer Security) = ఆ encryption చేసే protocol. (పాత పేరు **SSL** — ఇప్పుడు deprecated, కానీ జనాలు ఇంకా "SSL certificate" అంటారు. నిజానికి TLS 1.2/1.3.)

Real-life Scenario

> **HTTPS = రహస్య ఉత్తరాలు + గుర్తింపు కార్డు.** నువ్వు bank కి ఒక secret message పంపాలి:
>
> - **HTTP (insecure):** ఒక postcard మీద నీ password రాసి పంపడం. దారిలో postman, sorting staff — అందరూ చదవొచ్చు. 😱
> - **HTTPS (secure):** message ని ఒక **locked box** లో పెట్టి పంపడం. తాళం bank దగ్గరే ఉంది — దారిలో ఎవరూ తెరవలేరు (**confidentiality**). Box tamper అయితే తెలుస్తుంది (**integrity**). పైగా box మీద bank యొక్క **అధికారిక ముద్ర (certificate)** — నిజంగా bank కేనా అని నిర్ధారణ (**authentication**).
>
> **కీలక సవాలు:** locked box పంపాలంటే ముందు తాళం-చెవి (key) ఎలా share చేసుకోవాలి? చెవిని కూడా postcard లో పంపితే hacker దాన్ని కూడా చూస్తాడు! ఈ "key exchange" సమస్యని TLS handshake తెలివిగా solve చేస్తుంది (కింద).

Symmetric vs Asymmetric Encryption (పునాది)

TLS అర్థం కావాలంటే ఈ రెండు రకాల encryption తెలియాలి:

అంశం	Symmetric	Asymmetric (Public-key)
Keys	**ఒకే key** (encrypt + decrypt)	**రెండు keys** (public + private jodi)
వేగం	చాలా fast	నెమ్మది (heavy math)
సమస్య	Key ని ఎలా share చేయాలి? (insecure channel లో)	Slow, పెద్ద data కి కాదు
ఉదాహరణ	AES	RSA, ECC
analogy	ఒకే తాళం-చెవి ఇద్దరికీ	Public = అందరికీ ఇచ్చే lock, Private = నీ దగ్గరే ఉండే key

Asymmetric magic: Public key తో encrypt చేసినది private key తోనే decrypt అవుతుంది (vice versa). Public key అందరికీ ఇవ్వొచ్చు, private key రహస్యం. అంటే — ఎవరైనా నీ public key తో message encrypt చేసి పంపితే, నీ private key ఉన్న నువ్వు మాత్రమే చదవగలవు. **Key ని share చేయాల్సిన అవసరం లేకుండా secure communication!**

TLS తెలివి — రెండింటినీ కలపడం:

- **సమస్య:** Symmetric fast కానీ key share కష్టం. Asymmetric key share solve చేస్తుంది కానీ slow.
- **పరిష్కారం: Asymmetric ని వాడి ఒక symmetric key ని securely share చేసుకో. తర్వాత fast symmetric తో actual data encrypt చెయ్యి.** Best of both worlds. ఇదే TLS handshake చేసేది.

Certificates, CA & PKI — "నువ్వు నిజంగా GitHub వేనా?"

సమస్య: నీ browser github.com public key అందుకుంది. కానీ అది నిజంగా github దేనా, లేక hacker మధ్యలో తన public key ఇచ్చాడా (man-in-the-middle)? ఎలా నమ్మాలి?

పరిష్కారం — Certificate + Certificate Authority (CA):

- **SSL/TLS Certificate:** ఒక digital document — "ఈ public key నిజంగా github.com దే" అని చెప్పేది. దీన్ని ఒక **CA (Certificate Authority)** — నమ్మకమైన third party (DigiCert, Let's Encrypt) — **digitally sign** చేస్తుంది.
- **PKI (Public Key Infrastructure):** ఈ మొత్తం నమ్మకపు వ్యవస్థ — CAs, certificates, keys.
- **Chain of trust:** నీ browser/OS లో ముందే కొన్ని **root CAs** ని నమ్ముతూ store చేసి ఉంటుంది. github certificate ని ఆ trusted CA sign చేసింది కాబట్టి, browser దాన్ని నమ్ముతుంది.

```
Root CA (browser నమ్మేది, ముందే installed)
      │ signs
      ▼
Intermediate CA
      │ signs
      ▼
github.com యొక్క certificate  ← browser: "trusted CA sign చేసింది, నమ్ముతా ✓"
```

> **Analogy — passport & government.** నీ passport నిజమైనదని ఎలా నమ్ముతారు? దాన్ని **government (CA)** issue చేసింది, official seal ఉంది. Airport (browser) government ని నమ్ముతుంది కాబట్టి నీ passport ని నమ్ముతుంది. ఎవరైనా fake passport చేస్తే — government seal ఉండదు → reject. **Certificate = website యొక్క passport, CA = government.**

MERN connection: నువ్వు site deploy చేసేటప్పుడు **Let's Encrypt** (ఉచిత CA) నుండి TLS certificate తీసుకుంటావు. లేకపోతే browser "Not Secure" చూపిస్తుంది. `https://` పక్కన lock icon = valid certificate.

TLS Handshake — steps (అత్యంత ముఖ్యం)

Actual HTTP data పంపేముందు, TLS handshake జరుగుతుంది (TCP handshake తర్వాత). TLS 1.2 ని అర్థం చేసుకుందాం (concept clear అవుతుంది):

```
Client (browser)                        Server (github)
      │                                       │
```

```
│ ── 1. Client Hello ───────────►     │ "నా TLS versions, cipher suites,
│    (supported ciphers, random)      │  random number"
│                                     │
│ ◄── 2. Server Hello + Certificate ── │ "ఈ cipher వాడదాం, నా certificate
│    (chosen cipher, cert, random)    │  (public key లోపల), random"
│                                     │
│  3. Certificate verify              │ browser: CA sign check ✓
│    (browser CA chain చూస్తుంది)     │ "నిజంగా github! ✓"
│                                     │
│ ── 4. Key exchange ───────────►     │ browser ఒక "pre-master secret"
│    (server public key తో encrypt)   │ ని server public key తో encrypt
│                                     │ చేసి పంపుతుంది (server private
│                                     │ key తోనే తెరవగలదు)
│                                     │
│  5. ఇద్దరూ session key derive       │ ఇద్దరూ same SYMMETRIC key
│    (pre-master + randoms నుండి)     │ తయారుచేసుకుంటారు
│                                     │
│ ══ 6. encrypted data (symmetric) ══ │ ఇప్పుడు fast symmetric
│    GET /users (encrypted)           │ encryption తో actual data
```

Steps సారాంశం:

1. **Client Hello:** browser "నేను ఈ TLS versions, cipher suites support చేస్తా" + random number.
2. **Server Hello + Certificate:** server cipher ఎంచుకుని, తన **certificate** (public key తో) పంపుతుంది + random.
3. **Certificate verify:** browser CA chain చూసి "నిజమైన github యేనా?" నిర్ధారిస్తుంది.
4. **Key exchange:** browser ఒక secret ని server **public key తో encrypt** చేసి పంపుతుంది. Server **private key తోనే** దాన్ని తెరవగలదు (asymmetric).
5. **Session key:** ఇద్దరూ ఆ secret + randoms నుండి **same symmetric session key** derive చేసుకుంటారు.
6. **Encrypted data:** ఇక నుండి అంతా **fast symmetric encryption** తో. (Asymmetric ని key exchange కి మాత్రమే వాడారు.)

TLS 1.3 మెరుగుదల: Handshake ని **1 round-trip (1-RTT)** కి తగ్గించింది (TLS 1.2 కి 2-RTT). పాత insecure ciphers తీసేసింది. Faster + secure. (0-RTT resumption కూడా ఉంది.)

HTTP vs HTTPS

అంశం	HTTP	HTTPS
Port	80	443
Encryption	లేదు (plain text)	ఉంది (TLS)
Security	ఎవరైనా చదవొచ్చు, tamper చేయొచ్చు	Confidential + integrity + authentication
Certificate	అవసరం లేదు	కావాలి (CA issued)
వేగం	కొద్దిగా fast (handshake లేదు)	Handshake overhead కానీ TLS 1.3 తో negligible
SEO/browser	"Not Secure" warning	Lock icon, SEO boost

Advanced TLS concepts (SSE bonus)

- **Cipher Suite:** TLS లో వాడే algorithms combo — key exchange (ECDHE) + authentication (RSA) + symmetric encryption (AES-256-GCM) + hash (SHA-384). ఉదా `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`. Handshake లో client/server ఒక common cipher suite ఎంచుకుంటారు.
- **Forward Secrecy (PFS):** ప్రతి session కి కొత్త ephemeral key (ECDHE వల్ల). అంటే server private key future లో leak అయినా, **పాత recorded sessions ని decrypt చేయలేరు** — ప్రతి session key స్వతంత్రం. Modern TLS లో ముఖ్యం.
- **HSTS (HTTP Strict Transport Security):** `Strict-Transport-Security` header — "ఈ site ని ఎప్పుడూ HTTPS లోనే open చెయ్యి" అని browser కి చెప్తుంది. మొదటిసారి http→https downgrade attack ని ఆపుతుంది.
- **Self-signed vs CA certificate:** Self-signed = నువ్వే sign చేసుకున్నది (dev/testing కి, browser warning చూపిస్తుంది). CA-signed = trusted CA (Let's Encrypt) issue చేసినది (production, no warning).
- **SNI (Server Name Indication):** ఒకే IP మీద multiple HTTPS sites host చేసినప్పుడు, "ఏ site కావాలో" client hello లో చెప్పే extension — server సరైన certificate పంపడానికి.
- **mTLS (mutual TLS):** సాధారణ TLS లో client మాత్రమే server ని verify చేస్తుంది. mTLS లో **రెండు వైపులా** verify (server కూడా client certificate చూస్తుంది) — microservices, zero-trust security లో వాడతారు.

Key Points

- **HTTPS = HTTP + TLS.** 3 guarantees: **Confidentiality** (encrypt), **Integrity** (tamper detect), **Authentication** (certificate).
- **Symmetric = ఒకే key, fast** (AES). **Asymmetric = public+private jodi, slow** (RSA). TLS రెండింటినీ కలుపుతుంది.
- **TLS trick:** asymmetric తో symmetric key ని securely share → తర్వాత fast symmetric తో data. (Best of both.)
- **Certificate = website passport, CA = government** (trusted signer). Browser లో root CAs ముందే trusted. Chain of trust.
- **TLS handshake:** Client Hello → Server Hello+Cert → verify cert → key exchange (asymmetric) → session key → encrypted data (symmetric).
- **TLS 1.3 = 1-RTT** (faster than 1.2's 2-RTT), పాత ciphers తీసేసింది.
- **HTTPS port 443, HTTP port 80.**

Interview దృష్టి

- **"How does HTTPS work / TLS handshake?"** — Certificate verify → asymmetric తో symmetric key exchange → symmetric encryption. 3 guarantees (CIA). **Steps చెప్పగలగాలి.** Very common SSE question.
- **"Why use both symmetric and asymmetric?"** — Asymmetric slow కానీ secure key exchange; symmetric fast కానీ key share problem. TLS: asymmetric తో key share, symmetric తో data. Best of both.
- **"What is a certificate? How does browser trust it?"** — Certificate = public key + identity, CA signed. Browser root CAs ముందే నమ్ముతుంది → chain of trust. Fake cert = no valid CA signature → reject.
- **"HTTP vs HTTPS?"** — Port 80 vs 443, plain vs encrypted, no cert vs CA cert. HTTPS = CIA guarantees.
- **"What is a man-in-the-middle attack, how does HTTPS prevent it?"** — Attacker మధ్యలో traffic intercept. HTTPS: encryption (చదవలేడు) + certificate (fake server గుర్తించడం). Cert లేని/invalid site కి browser

warning.

- **Gotcha:** SSL vs TLS — SSL పాతది, insecure, deprecated. అందరూ ఇప్పుడు TLS 1.2/1.3 వాడతారు కానీ "SSL certificate" అని అలవాటుగా అంటారు.
- **Gotcha:** HTTPS server ని hack నుండి పూర్తిగా కాపాడదు — ఇది transit లో (transport) encryption మాత్రమే. Server లో data breach, XSS, SQL injection ఇంకా సాధ్యం. HTTPS ≠ "పూర్తి security".

---

10. DNS Deep — Hierarchy, Resolution, Record Types, Caching

వివరణ

DNS (Domain Name System) = internet యొక్క phonebook. మనుషులు `github.com` గుర్తుంచుకుంటారు, కానీ computers **IP addresses** (140.82.112.3) తోనే మాట్లాడతాయి. DNS ఈ **name → IP translation** చేస్తుంది. ప్రతి `fetch("https://github.com")` , ప్రతి URL type — మొదట DNS lookup జరుగుతుంది, తర్వాతే మిగతాది.

ఎందుకు అవసరం? IP addresses గుర్తుంచుకోవడం కష్టం (140.82.112.3 vs github.com). పైగా IPs మారొచ్చు (server మారితే) కానీ domain name అలాగే ఉంటుంది. DNS ఈ layer of indirection ఇస్తుంది.

Real-life Scenario

> **DNS = phone contacts + telephone enquiry (180-directory).** నీకు friend పేరు "Ravi" గుర్తుంది, phone number గుర్తు లేదు. నువ్వు contacts లో "Ravi" వెతికి number పొందుతావు (name → number). Contacts లో లేకపోతే enquiry service కి call చేసి అడుగుతావు.
>
> DNS సరిగ్గా ఇదే: నీకు "github.com" గుర్తుంది, IP గుర్తు లేదు. నీ computer DNS ని అడిగి IP పొందుతుంది. ఒకసారి పొందాక **cache** (contacts) లో save చేసుకుంటుంది — మళ్ళీ మళ్ళీ అడగకుండా. **Names మనుషులకి, numbers computers కి — DNS మధ్య translator.**

DNS Hierarchy — నిర్మాణం

DNS ఒక **distributed, hierarchical** system — ఒక్క server కాదు, వేల servers ఒక tree లా:

```
                 . (Root)          ← 13 root server clusters (worldwide)
                 │
      ┌──────────┼──────────┐
     .com       .org       .in     ← TLD (Top-Level Domain) servers
      │
    github         ← Authoritative name server (github యొక్క actual records)
      │
   ┌──┴──┐
  www    api    ← subdomains
```

Domain name ని కుడి నుండి ఎడమకి చదవాలి:

```
   www  .  github  .  com  .
   └─┬┘    └──┬──┘    └┬┘   └ root (implicit dot)
```

```
  subdomain   domain   TLD
```

స్థాయి	ఏమిటి	ఉదాహరణ	పని
Root	tree మూలం	`.`	TLD servers ఎక్కడో చెప్తుంది
TLD	Top-Level Domain	`.com` , `.org` , `.in` , `.io`	ఆ domain యొక్క authoritative server చెప్తుంది
Authoritative	Domain యొక్క నిజమైన records	github.com's server	actual IP address ఇస్తుంది
Subdomain	domain లో భాగం	`www` , `api` , `blog`	వేర్వేరు services

DNS Resolution — steps (interview must-know)

నువ్వు `github.com` type చేస్తే, IP కనుక్కోవడానికి ఈ steps జరుగుతాయి:

```
నీ browser: "github.com యొక్క IP ఏమిటి?"
      │
      ▼
┌─────────────────────────────────────────────┐
│ 0. Browser cache → OS cache → చూస్తుంది        │  (ముందు local)
└─────────────────────────────────────────────┘
      │ (cache miss అయితే)
      ▼
1. Recursive Resolver (నీ ISP/8.8.8.8) ← ఇది అన్నీ చేస్తుంది
      │
      ├──► 2. Root server: "github.com?"
      │      Root: ".com కోసం ఈ TLD server అడుగు"
      │
      ├──► 3. TLD (.com) server: "github.com?"
      │      TLD: "github యొక్క authoritative server ఇది"
      │
      ├──► 4. Authoritative server: "github.com?"
      │      Auth: "IP = 140.82.112.3" ✓
      │
      ▼
5. Resolver → browser కి IP ఇస్తుంది (+ cache లో save)
      │
      ▼
browser ఇప్పుడు 140.82.112.3 కి TCP connect చేస్తుంది
```

రెండు రకాల queries:

- **Recursive query:** browser → resolver. "నాకు final answer తీసుకురా" (resolver మొత్తం పని చేస్తుంది).
- **Iterative query:** resolver → root → TLD → authoritative. ప్రతి server "పూర్తి జవాబు తెలియదు, తర్వాత ఎవరిని అడగాలో చెప్తా" (referral).

> **Analogy — కొత్త ఊరిలో address వెతకడం.** నువ్వు కొత్త నగరంలో ఒక ఇల్లు వెతుకుతున్నావు. (1) నగర ప్రవేశంలో ఒకరిని అడిగావు (root) → "ఆ area కి వెళ్ళు" అన్నారు. (2) Area లో అడిగావు (TLD) → "ఆ colony కి వెళ్ళు". (3) Colony లో అడిగావు (authoritative) → "అదిగో ఆ ఇల్లు" (IP). ప్రతి ఒక్కరూ **తర్వాతి step** మాత్రమే చెప్పారు, పూర్తి address కాదు — ఇదే iterative resolution.

DNS Record Types — ముఖ్యమైనవి

DNS లో వేర్వేరు రకాల records ఉంటాయి (ఒక్క IP mapping కాదు):

Record	పని	ఉదాహరణ
A	Name → **IPv4** address	github.com → 140.82.112.3
AAAA	Name → **IPv6** address	github.com → 2606:50c0::153
CNAME	Name → **వేరే name** (alias)	www.site.com → site.com
MX	**Mail server** (email routing)	gmail.com → mail servers
NS	**Name server** (ఆ domain ఎవరు handle చేస్తారు)	github.com → ns1.github.com
TXT	Text data (verification, SPF, DKIM)	domain ownership proof
PTR	IP → Name (reverse lookup)	140.82.112.3 → github.com
SOA	Zone యొక్క authority info	admin, refresh times

MERN connection: నువ్వు domain కొని site deploy చేసేటప్పుడు — **A record** ని నీ server IP కి, **CNAME** (www → root), **MX records** (email కి) set చేస్తావు. Vercel/Netlify deploy చేసినప్పుడు వాళ్ళు "ఈ CNAME/A record add చెయ్యి" అని చెప్తారు — ఇదే.

A vs CNAME (interview trap): A record = నేరుగా IP కి points. CNAME = వేరే domain name కి points (alias). CNAME resolve అయ్యి, చివరికి ఒక A record దగ్గరకి చేరుతుంది. Root domain కి CNAME వాడకూడదు (technical limitation), A record వాడాలి.

DNS Caching & TTL

DNS ప్రతిసారి పూర్తి resolution చేస్తే చాలా slow. అందుకే **caching** — పొందిన IP ని కొంతసేపు గుర్తుంచుకుంటారు.

TTL (Time To Live): ప్రతి DNS record కి ఒక TTL (seconds) ఉంటుంది — "ఈ answer ని ఎంతసేపు cache చేయొచ్చు". TTL 3600 అంటే 1 గంట cache. తర్వాత మళ్ళీ fresh lookup.

```
Caching layers (ఏది ముందు చూస్తారో):
1. Browser cache      (chrome://net-internals)
2. OS cache           (n(నీ laptop))
3. Router cache       (నీ WiFi router)
4. Resolver cache     (ISP / 8.8.8.8)
   → ప్రతి level TTL వరకు గుర్తుంచుకుంటుంది
```

TTL trade-off (interview):

- **High TTL (ఉదా 24h):** fewer lookups, faster, servers మీద load తక్కువ. కానీ IP మారితే, పాత IP 24h వరకు cache లో ఉంటుంది → users పాత server కి వెళ్తారు.
- **Low TTL (ఉదా 60s):** IP మార్పు వెంటనే propagate. కానీ ఎక్కువ lookups.
- **Practical:** migration ముందు TTL తగ్గించి (60s), migrate చేసి, స్థిరపడ్డాక పెంచుతారు. "DNS propagation" ఆలస్యం అనేది ఈ caching వల్లే.

MERN gotcha: నువ్వు server IP మార్చాక "site పాత దానికే వెళ్తోంది!" అంటే — DNS cache. `dig github.com` లేదా `nslookup github.com` తో check చేయొచ్చు. Browser లో `chrome://net-internals/#dns` → clear cache.

DNS — advanced uses (system design లో ముఖ్యం)

DNS కేవలం name→IP కాదు — ఇది scalability కి ఒక powerful tool. SSE/system-design interview లో ఇవి అడుగుతారు:

- **DNS-based Load Balancing (Round-Robin DNS):** ఒక domain కి **అనేక A records** (multiple server IPs) ఇస్తారు. ప్రతి DNS query కి వేర్వేరు IP order లో ఇస్తారు → traffic servers మధ్య పంచబడుతుంది. Simple కానీ health-check ఉండదు (down server కి కూడా పంపొచ్చు).
- **GeoDNS:** User location ఆధారంగా **దగ్గరి server IP** ఇవ్వడం. India user → Mumbai server, US user → Virginia server. Latency తగ్గుతుంది.
- **CDN connection:** నువ్వు `cdn.site.com` అడిగితే, CDN (Cloudflare, Akamai) DNS నిన్ను **అత్యంత దగ్గరి edge server** కి route చేస్తుంది (GeoDNS + Anycast). నీ static assets (images, JS) అక్కడి నుండి fast గా వస్తాయి.
- **Anycast:** ఒకే IP address ని **అనేక చోట్ల (data centers)** advertise చేయడం. Network నిన్ను automatically దగ్గరి దానికి route చేస్తుంది. `8.8.8.8` (Google DNS), `1.1.1.1` (Cloudflare) Anycast — ప్రపంచమంతా same IP కానీ దగ్గరి server జవాబిస్తుంది.
- **Failover:** Primary server down అయితే, DNS ని backup server IP కి మార్చడం (low TTL తో fast switch).

> **Analogy — pizza chain యొక్క single phone number.** "1800-PIZZA" ఒక్క number, కానీ నువ్వు call చేస్తే **నీ దగ్గరి branch** కి connect అవుతుంది (Anycast/GeoDNS). Number same, కానీ location ఆధారంగా వేర్వేరు branch. అలాగే `1.1.1.1` ప్రపంచమంతా ఒకటే కానీ దగ్గరి data center జవాబిస్తుంది.

MERN connection: నీ React app ని Vercel/Netlify లో deploy చేస్తే, వాళ్ళ CDN + Anycast DNS వల్ల నీ site ప్రపంచమంతా fast. నీ API కి multiple regions ఉంటే, GeoDNS తో users దగ్గరి region కి వెళ్తారు.

Key Points

- **DNS = name → IP translation** ("internet phonebook"). ప్రతి URL ముందు DNS lookup.
- **Hierarchy:** Root (.) → TLD (.com) → Authoritative (github's server). కుడి నుండి ఎడమకి.
- **Resolution:** browser → recursive resolver → root → TLD → authoritative → IP (+ cache). Recursive (resolver మొత్తం చేస్తుంది) vs iterative (referrals).
- **Records:** A (IPv4), AAAA (IPv6), CNAME (alias/name), MX (mail), NS (name server), TXT (verification), PTR (reverse).

- **A = IP కి, CNAME = వేరే name కి** (alias). Root కి CNAME వాడొద్దు.
- **Caching + TTL:** పొందిన IP ని TTL seconds cache. High TTL = fast కానీ మార్పు slow; Low TTL = fresh కానీ ఎక్కువ lookups.
- DNS **UDP port 53** (fast); పెద్ద responses TCP.

Interview దృష్టి

- **"What happens in DNS resolution?"** — browser/OS cache → recursive resolver → root → TLD → authoritative → IP → cache. Steps చెప్పగలగాలి. (Topic 11 లో ఇది full journey లో భాగం.)
- **"Recursive vs iterative DNS query?"** — Recursive: client ఒక్క resolver ని అడిగి final answer పొందుతాడు. Iterative: resolver ప్రతి server ని అడిగి referrals follow అవుతుంది.
- **"A vs CNAME record?"** — A = name→IP నేరుగా. CNAME = name→వేరే name (alias). CNAME చివరికి A record కి resolve. Root domain కి CNAME వాడలేం.
- **"What is TTL in DNS?"** — Record ని ఎంతసేపు cache చేయొచ్చో. Trade-off: high = fast/less load కానీ మార్పు slow; low = fresh కానీ ఎక్కువ queries.
- **"Why is DNS distributed/hierarchical?"** — ఒక్క server = single point of failure + bottleneck (billions of queries). Hierarchy → load పంచడం, resilience, delegation (ప్రతి domain తన records manage).
- **"Which transport does DNS use?"** — UDP 53 (fast, small). Zone transfers / పెద్ద responses → TCP.
- **Gotcha:** "DNS propagation" నిజంగా propagation కాదు — పాత cached records TTL expire అవ్వడానికి పట్టే time. Cache concept.
- **Gotcha:** 8.8.8.8 (Google), 1.1.1.1 (Cloudflare) = public recursive resolvers. నీ ISP resolver బదులు వీటిని వాడొచ్చు (faster/privacy).

---

11. "URL enter చేస్తే ఏం జరుగుతుంది" — Full End-to-End Journey

వివరణ

ఇది **అత్యంత classic system-design/networking interview question** — "You type `https://github.com` in the browser and press Enter. Walk me through everything that happens until the page renders." ఈ ఒక్క ప్రశ్న నీకు CN మొత్తం తెలుసా అని test చేస్తుంది — DNS, TCP, TLS, HTTP, routing, rendering — అన్నీ ఇక్కడ కలుస్తాయి.

ఈ topic ప్రత్యేకం — ఇది కొత్త concept కాదు, **ఇప్పటివరకు నేర్చుకున్నవన్నీ ఒక కథలా (story) కలపడం.** ఈ story ని గట్టిగా పట్టుకో — interview లో నీళ్ళులా చెప్పగలగాలి.

Real-life Scenario

> **URL enter చేయడం = ఒక international courier పంపడం, మొదటి నుండి చివరి వరకు.** నువ్వు అమెరికాలో ఉన్న స్నేహితుడికి (github server) ఉత్తరం పంపి, జవాబు తెప్పించుకోవాలి:
>
> 1. అతని **address కనుక్కోవాలి** (DNS — పేరు నుండి location).
> 2. అతనితో **line establish చేయాలి** (TCP handshake — "మాట్లాడొచ్చా?").
> 3. **రహస్యంగా మాట్లాడటానికి secret code set చేయాలి** (TLS handshake — locked box).
> 4. **అసలు message పంపాలి** (HTTP request — "నీ homepage కావాలి").
> 5. అతను **జవాబు పంపుతాడు** (HTTP response — HTML).
> 6. నువ్వు ఆ జవాబుని **అర్థం చేసుకుని బొమ్మ గీసుకుంటావు** (browser rendering).
>
> ఇదే మొత్తం journey. ప్రతి step వెనుక ఒక CN concept ఉంది. ఇప్పుడు detail గా.

The Full Journey — Step by Step

```
  నువ్వు type చేసి Enter నొక్కావు: https://github.com
         │
         ▼
┌──────────────────────────────────────────────────────────────┐
│ STEP 1: URL Parsing (browser)                                │
│   scheme=https, host=github.com, port=443 (default), path=/  │
├──────────────────────────────────────────────────────────────┤
│ STEP 2: DNS Resolution (name → IP)                           │
│   browser cache → OS cache → resolver → root → TLD → auth    │
│   github.com → 140.82.112.3                                  │
├──────────────────────────────────────────────────────────────┤
│ STEP 3: TCP Connection (3-way handshake)                     │
│   SYN → SYN-ACK → ACK  (port 443)                            │
├──────────────────────────────────────────────────────────────┤
│ STEP 4: TLS Handshake (HTTPS encryption)                     │
│   Client Hello → Server Hello+Cert → verify → key exchange   │
│   → symmetric session key                                    │
├──────────────────────────────────────────────────────────────┤
│ STEP 5: HTTP Request                                         │
│   GET / HTTP/2  Host: github.com  (encrypted via TLS)        │
├──────────────────────────────────────────────────────────────┤
│ STEP 6: Server Processing                                    │
│   load balancer → app server → DB → HTML తయారు               │
├──────────────────────────────────────────────────────────────┤
│ STEP 7: HTTP Response                                        │
│   200 OK + HTML body (encrypted)                             │
├──────────────────────────────────────────────────────────────┤
│ STEP 8: Browser Rendering                                    │
│   parse HTML → CSS/JS fetch (మళ్ళీ steps 2-7) → paint         │
└──────────────────────────────────────────────────────────────┘
         │
         ▼
  Page కనిపిస్తుంది! 🎉
```

ప్రతి Step లోతుగా

STEP 1 — URL Parsing. Browser URL ని విడగొడుతుంది: **scheme** (`https`), **host** (`github.com`), **port** (default 443 for https), **path** (/), query params, fragment. HSTS check (ఈ site HTTPS-only అని ముందే తెలిస్తే http→https). నెట్‌వర్క్ కి వెళ్ళేముందు browser cache కూడా చూస్తుంది (ఈ page ఇప్పటికే cache లో ఉందా?).

STEP 2 — DNS Resolution. (Topic 10) Host name ని IP గా మార్చాలి:

- ముందు **browser cache** → **OS cache (hosts file)** → **router** → **ISP resolver** – cache లో ఉంటే వెంటనే.
- లేకపోతే recursive resolver → **root** → **TLD (.com)** → **authoritative** → IP (140.82.112.3).
- ఇది UDP port 53. TTL ప్రకారం cache అవుతుంది.

STEP 3 — TCP Connection (3-way handshake). (Topic 6) IP దొరికాక, ఆ server తో reliable connection:

- **SYN** → **SYN-ACK** → **ACK.** Destination port 443 (HTTPS).
- ఇక్కడ నుండి packets IP layer ద్వారా routers గుండా (topic 5) hop-by-hop ప్రయాణిస్తాయి. నీ WiFi router (default gateway) → ISP → backbone → github data center.

STEP 4 — TLS Handshake. (Topic 9) HTTPS కాబట్టి encryption setup:

- **Client Hello** → **Server Hello + Certificate** → **certificate verify (CA chain)** → **key exchange (asymmetric)** → **symmetric session key.**
- ఇప్పటినుండి అంతా encrypted. (TLS 1.3 అయితే 1-RTT – fast.)

STEP 5 — HTTP Request. (Topic 8) ఇప్పుడు actual request:

```
GET / HTTP/2
Host: github.com
User-Agent: Mozilla/5.0...
Accept: text/html
Cookie: session=... (ఉంటే)
```

ఇది TLS ద్వారా encrypt అయ్యి, TCP segments గా, IP packets గా (encapsulation, topic 3) server కి వెళ్తుంది.

STEP 6 — Server Processing. Server side (నీ MERN backend లాంటిదే):

- **Load balancer** request ని ఒక app server కి route చేస్తుంది.
- App server (Node/Express లాంటిది) route match చేసి, **database** query చేసి, HTML/JSON తయారు చేస్తుంది.
- Caching (Redis, CDN) ఉంటే అక్కడి నుండి fast.

STEP 7 — HTTP Response. (Topic 8) Server జవాబు:

```
HTTP/2 200 OK
Content-Type: text/html
Set-Cookie: ...
Cache-Control: ...

<!DOCTYPE html>...
```

ఇది మళ్ళీ TLS encrypt → TCP → IP → నీ browser కి (decapsulation).

STEP 8 — Browser Rendering. (ఇది CN కాదు కానీ interview లో అడుగుతారు):

```
HTML parse → DOM tree
CSS parse → CSSOM tree
DOM + CSSOM → Render tree
Layout (ఏది ఎక్కడ) → Paint (pixels) → Composite
```

- HTML లో `<img>` , `<script>` , `<link rel=stylesheet>` కనిపిస్తే — **ప్రతి దానికి మళ్ళీ steps 2-7** (DNS/TCP అవసరమైతే, HTTP fetch). HTTP/2 multiplexing వల్ల ఇవి parallel.
- JS execute అవుతుంది (నీ React app hydrate అవుతుంది). `fetch()` calls ఉంటే మళ్ళీ API requests (steps 2-7).

ఇక్కడ ప్రతి Layer ఎలా వాడబడింది (సారాంశం)

Step	ఏ concept	ఏ layer
DNS resolution	Name → IP	Application (DNS), UDP
TCP handshake	Reliable connection	Transport (L4)
Routing (packets)	Hop-by-hop, IP	Network (L3)
MAC/frames (WiFi)	పక్క device కి	Data Link (L2)
TLS handshake	Encryption	Presentation/App
HTTP request/response	Actual data	Application (L7)
Encapsulation	ప్రతి layer header	అన్ని layers

ఇదే ఈ guide యొక్క పరాకాష్ఠ (climax): నీ ఒక్క `fetch()` లేదా URL enter — DNS (topic 10) + TCP (topic 6) + IP routing (topic 5) + Data Link (topic 4) + TLS (topic 9) + HTTP (topic 8) + encapsulation (topic 3) — అన్నీ కలిసి పని చేస్తాయి. ఇప్పుడు నీకు ఆ 200ms లో ఏం జరిగిందో పూర్తిగా తెలుసు.

సంక్షిప్త mnemonic — "డ్ TCP TLS హ్ రెండర్"

Interview లో order మర్చిపోకుండా: **"Parse → DNS → TCP → TLS → HTTP → Server → Response → Render."** ఒక్క వాక్యం: **"URL parse చేసి, DNS తో IP కనుక్కుని, TCP connect చేసి, TLS తో encrypt చేసి, HTTP request పంపి, server process చేసి, response తీసుకుని, browser render చేస్తుంది."**

Key Points

- **8 steps:** URL parse → DNS → TCP handshake → TLS handshake → HTTP request → server processing → HTTP response → browser render.
- **DNS (topic 10):** cache → resolver → root → TLD → authoritative → IP.
- **TCP (topic 6):** SYN/SYN-ACK/ACK, port 443.

- **TLS (topic 9):** Hello, certificate, key exchange, symmetric session.
- **HTTP (topic 8):** GET request, 200 response with HTML.
- **Render:** HTML→DOM, CSS→CSSOM, render tree, layout, paint. Sub-resources (CSS/JS/img) → steps 2-7 మళ్ళీ.
- **ఇది CN మొత్తానికి integration** — ప్రతి layer ఇక్కడ ఒక పని చేస్తుంది.

Interview దృష్టి

- **"What happens when you type a URL and press Enter?"** — 8 steps నీళ్ళులా చెప్పు. Interviewer ఏ step లోనైనా "ఇక్కడ deep గా చెప్పు" అని అడగొచ్చు (DNS steps? TCP handshake? TLS?) — కాబట్టి ప్రతి step వెనుక topic గట్టిగా ఉండాలి. **ఇది #1 asked networking question.**
- **Follow-up: "DNS ఎలా resolve అవుతుంది?"** → topic 10 steps.
- **Follow-up: "TCP handshake ఎందుకు?"** → reliable connection, topic 6.
- **Follow-up: "HTTPS ఎలా secure?"** → TLS handshake, topic 9.
- **Follow-up: "Page ఎలా render అవుతుంది?"** → DOM/CSSOM/render tree/paint.
- **Follow-up: "సైట్ ఎందుకు slow?"** — DNS slow, TCP/TLS handshake latency (RTT), large payloads, render-blocking CSS/JS, no caching/CDN. Optimizations: DNS prefetch, keep-alive, HTTP/2/3, CDN, caching, compression.
- **Gotcha:** ఏ step మర్చిపోకు — చాలామంది TLS లేదా encapsulation మర్చిపోతారు. Order: **Parse, DNS, TCP, TLS, HTTP, Render.**
- **Gotcha:** "cache" ప్రతిచోటా — browser cache, DNS cache, TCP connection reuse (keep-alive), CDN cache. Senior candidate వీటిని mention చేస్తాడు.

Part 4 — Reference (Security & Interview)

> చివరి Part — **practical security concepts + tools + interview preparation.** Topic 12 లో network security (firewall, VPN, proxy), common attacks, మరియు నువ్వు రోజూ terminal లో వాడగల commands. Topic 13 లో మొత్తం guide ని ఒక **interview-ready cheat sheet** గా — rapid-fire Q&A, memory tricks, common mistakes. ఇది చదివి, interview ముందు revise చేయడానికి perfect.

---

12. Network Security & Tools Basics

వివరణ

SSE గా నీకు security fundamentals తెలియాలి — నీ APIs ని ఎవరు, ఎలా attack చేయగలరు, ఎలా protect చేయాలి. అలాగే network problems debug చేయడానికి కొన్ని **tools/commands** — ఇవి interview లోనూ, job లోనూ అడుగుతారు. ఈ topic reference-heavy — గుర్తుంచుకోవాల్సినవి table లుగా ఇస్తున్నాను.

Firewall — network యొక్క security guard

Firewall = ఏ traffic లోపలికి/బయటికి వెళ్ళాలో నియంత్రించే filter (rules ఆధారంగా). "Allow port 443, block port 23" లాంటి rules.

> **Analogy — apartment security guard.** ప్రతి visitor ని guard చెక్ చేస్తాడు — "ఎవరు? ఎక్కడికి? అనుమతి ఉందా?" List లో ఉంటేనే లోపలికి. Firewall కూడా అంతే — ప్రతి packet ని rules తో check చేసి allow/block చేస్తుంది.

- **Packet-filtering firewall:** IP/port/protocol చూసి allow/block (basic).
- **Stateful firewall:** connection state గుర్తుంచుకుంటుంది (ఈ packet ఒక valid connection లో భాగమా?).
- **Application firewall (WAF):** HTTP content చూస్తుంది (SQL injection, XSS block — నీ web app కి).
- **MERN:** నీ server లో సాధారణంగా port 443 (HTTPS), 22 (SSH) మాత్రమే open, మిగతావి firewall block. Security groups (AWS) ఒక firewall.

VPN — Virtual Private Network

VPN = internet మీద ఒక encrypted "tunnel" — నీ traffic ని encrypt చేసి, ఒక VPN server గుండా route చేస్తుంది. దీనివల్ల నీ real IP దాక్కుంటుంది, traffic ఎవరూ చూడలేరు.

> **Analogy — public రోడ్డు మీద private surangam (tunnel).** అందరూ చూసే highway (internet) మీద నీ కోసం ఒక సొరంగం. బయటివారికి నువ్వు లోపల ఏం మోసుకెళ్తున్నావో కనిపించదు, ఎక్కడికి వెళ్తున్నావో తెలియదు — VPN server దగ్గర మాత్రమే బయటికి వస్తావు.

- **వాడకం:** remote work (company network ని securely access), privacy (ISP నీ traffic చూడకుండా), geo-restrictions bypass.
- **ఎలా:** నీ device ↔ VPN server మధ్య encrypted tunnel. Website కి నీ IP కాదు, VPN server IP కనిపిస్తుంది.

Proxy Server — మధ్యవర్తి

Proxy = client మరియు server మధ్య నిలబడే intermediary. Requests ని forward చేస్తుంది.

రకం	ఎవరి కోసం	పని
Forward proxy	Client కోసం	Client → proxy → internet. Client IP దాచడం, filtering, caching (office proxy).
Reverse proxy	Server కోసం	Internet → proxy → backend servers. Load balancing, SSL termination, caching (**Nginx, Cloudflare**).

VPN vs Proxy (interview): Proxy ఒక **application** traffic ని route చేస్తుంది (ఉదా browser), encryption లేదు (సాధారణంగా). VPN **మొత్తం device traffic** ని encrypt చేసి route చేస్తుంది. VPN = full tunnel + encryption; Proxy = single-app forwarding.

MERN: Nginx ఒక reverse proxy గా నీ Node app ముందు నిలబడి — SSL terminate చేయడం, static files serve చేయడం, load balance చేయడం. నీ deployment లో ఇది common.

Load Balancer — L4 vs L7 (SSE/system-design favorite)

Load Balancer = incoming traffic ని అనేక backend servers మధ్య పంచే device/software (ఒక server overload అవ్వకుండా, scale చేయడానికి). ఇది ఒక ప్రత్యేక reverse proxy. SSE interview లో "ఎలా scale చేస్తావు?" అంటే load balancer ముఖ్యం.

> **Analogy — bank లో token/queue manager.** Bank లో 5 counters (servers) ఉన్నాయి. ఒక్క queue అయితే counter-1 దగ్గర జనం, మిగతావి ఖాళీ. **Queue manager (load balancer)** ప్రతి customer ని ఖాళీగా ఉన్న counter కి పంపుతాడు — అందరూ సమానంగా busy, wait time తక్కువ. ఒక counter మూసేస్తే (server down), అతను మిగతా counters కి మాత్రమే పంపుతాడు (health check).

L4 vs L7 load balancer (కీలక తేడా):

అంశం	L4 (Transport)	L7 (Application)
ఏ layer	Layer 4 (TCP/UDP)	Layer 7 (HTTP)
దేని ఆధారంగా route	IP + port మాత్రమే	URL, headers, cookies, content
Content చూస్తుందా?	లేదు (packet లోపల చూడదు)	అవును (HTTP request చదువుతుంది)
వేగం	చాలా fast (simple)	కొద్దిగా slow (parse చేయాలి)
సామర్థ్యం	Basic distribution	Smart routing (`/api` →A, `/img` →B)
ఉదాహరణ	AWS NLB, HAProxy (TCP mode)	AWS ALB, Nginx, Cloudflare

- **L4:** packet యొక్క IP/port చూసి, content చూడకుండా forward. Fast కానీ "dumb" — ఏ URL అనేది తెలియదు.
- **L7:** HTTP request చదివి, **path/header/cookie ఆధారంగా** route చేస్తుంది. ఉదా `/api/*` → backend servers, `/static/*` → CDN, mobile users → వేరే pool. SSL termination, sticky sessions కూడా ఇక్కడ.

Load balancing algorithms (ఎలా పంచుతారు):

- **Round Robin:** వరుసగా ఒక్కో server కి (1,2,3,1,2,3...).
- **Least Connections:** అత్యంత తక్కువ active connections ఉన్న server కి.
- **IP Hash:** client IP ఆధారంగా (same client → same server, sticky sessions కి).
- **Weighted:** బలమైన server కి ఎక్కువ traffic.

Health checks: load balancer ప్రతి server కి periodic ping/HTTP check పంపి, down అయిన server ని pool నుండి తీసేస్తుంది (traffic పంపదు). ఇది **high availability** ఇస్తుంది.

MERN connection: నీ Node app ని multiple instances లో run చేసి (PM2 cluster, Kubernetes pods), ముందు Nginx/ALB load balancer పెడితే — traffic పంచబడుతుంది, ఒకటి crash అయినా మిగతావి serve చేస్తాయి (horizontal scaling). WebSockets వాడితే sticky sessions (IP hash) కావాలి.

Common Attacks — తెలియాల్సినవి

Attack	ఏమిటి	రక్షణ
DDoS	Distributed Denial of Service — వేల systems నుండి fake traffic పంపి server ని ముంచెత్తడం (crash)	Rate limiting, CDN (Cloudflare), auto-scaling, traffic filtering
MITM	Man-in-the-Middle — attacker మధ్యలో traffic intercept/modify	**HTTPS/TLS** (encryption + certificate), HSTS, VPN
Phishing	Fake site/email తో credentials దొంగిలించడం	User awareness, 2FA, email filtering
SQL Injection	Input ద్వారా malicious SQL — DB hack	Parameterized queries, ORM, input validation
XSS	Cross-Site Scripting — malicious JS inject	Input sanitize, `HttpOnly` cookies, CSP header
CSRF	Cross-Site Request Forgery — user తెలియకుండా actions	CSRF tokens, `SameSite` cookies
DNS Spoofing	Fake DNS response తో తప్పు IP కి పంపడం	DNSSEC, trusted resolvers
Port Scanning	Open ports వెతకడం (attack ముందు)	Firewall, close unused ports

DDoS deep (interview favorite): ఒక్క system attack = DoS. వేల compromised systems (botnet) కలిసి attack = **DDoS** (distributed). Server real traffic handle చేయలేక down. రక్షణ: **rate limiting** (per-IP limits), **CDN/WAF** (Cloudflare fake traffic absorb), **auto-scaling**, **traffic scrubbing**.

MITM & HTTPS connection: Topic 9 లో చూశాం — MITM ని HTTPS ఆపుతుంది. Attacker traffic చూసినా encrypted (చదవలేడు), fake server పెట్టినా certificate invalid (browser warning). అందుకే public WiFi లో HTTPS లేని sites ప్రమాదకరం.

Useful Commands — terminal tools

నువ్వు నేర్చుకోవాల్సిన debugging commands (interview లో "ఈ problem ఎలా debug చేస్తావు?" అని అడిగినప్పుడు వీటిని mention చెయ్యి):

Command	పని	ఉదాహరణ
ping	Server reachable ఉందా, latency (ICMP)	`ping google.com`
traceroute / tracert	Packet ఏ ఏ routers (hops) గుండా వెళ్తుందో	`traceroute github.com`
nslookup / dig	DNS lookup (name → IP, records)	`dig github.com` , `nslookup github.com`
netstat	Open connections, listening ports	`netstat -an` , `netstat -tulpn`
curl	HTTP requests terminal నుండి	`curl -I https://github.com`
ifconfig / ip addr	నీ IP, network interfaces	`ifconfig` , `ip addr`
telnet / nc	Port open ఉందా test	`nc -zv github.com 443`
ss	Socket statistics (netstat modern)	`ss -tulpn`

ఉదాహరణలు — నిజంగా ఏం చూపిస్తాయి:

```
# 1. Server reachable + latency
$ ping github.com
64 bytes from 140.82.112.3: icmp_seq=1 ttl=52 time=45.2 ms

# 2. ఏ దారిలో వెళ్తోంది (ప్రతి hop = ఒక router)
$ traceroute github.com
 1  192.168.1.1   1.2 ms      ← నీ router (default gateway)
 2  10.0.0.1      5.4 ms      ← ISP
 3  ...          20.1 ms      ← backbone
 8  140.82.112.3 45.2 ms      ← github (destination)

# 3. DNS lookup
$ dig github.com +short
140.82.112.3

# 4. HTTP headers మాత్రమే (body కాదు)
$ curl -I https://github.com
HTTP/2 200
content-type: text/html; charset=utf-8
cache-control: max-age=0, private, must-revalidate

# 5. ఏ ports listening (నీ Node server run అవుతోందా?)
$ netstat -an | grep 3000
tcp4  0  0  *.3000   *.*   LISTEN     ← port 3000 open ✓
```

Debugging workflow (interview): "API పని చేయట్లేదు" అంటే —

1. `ping` — server reachable ఉందా?
2. `dig` — DNS సరిగ్గా resolve అవుతోందా?
3. `curl -I` — server responds అవుతోందా, ఏ status code?
4. `netstat` / `ss` — server port listening ఉందా?
5. `traceroute` — ఎక్కడ packet ఆగుతోంది?

Key Points

- **Firewall = traffic filter** (rules ఆధారంగా allow/block). Packet-filtering, stateful, WAF (app-level).
- **VPN = encrypted tunnel** (full device traffic, IP hide). **Proxy = intermediary** (forward: client-side; reverse: server-side, Nginx/Cloudflare).
- **VPN vs Proxy:** VPN full+encrypted, Proxy single-app forwarding.
- **Load Balancer:** traffic ని servers మధ్య పంచడం (scale + HA). **L4** = IP/port (fast, dumb), **L7** = HTTP content/URL (smart). Algorithms: round robin, least connections, IP hash. Health checks.
- **DDoS** = botnet flood (rate limit, CDN, scale). **MITM** = intercept (HTTPS/TLS ఆపుతుంది). XSS/CSRF/SQLi = web attacks.
- **Commands:** ping (reachability), traceroute (path/hops), dig/nslookup (DNS), curl (HTTP), netstat/ss (ports/connections).
- **Debug workflow:** ping → dig → curl → netstat → traceroute.

Interview దృష్టి

- **"VPN vs Proxy?"** — VPN = full device traffic, encrypted tunnel, IP hide. Proxy = per-app forwarding, encryption అవసరం లేదు. Reverse proxy (Nginx) = server-side (load balance, SSL).
- **"What is DDoS, how to prevent?"** — Distributed flood from botnet → server overwhelmed. Prevent: rate limiting, CDN/WAF (Cloudflare), auto-scaling, traffic filtering.
- **"How does HTTPS prevent MITM?"** — Encryption (intercept చేసినా చదవలేడు) + certificate (fake server గుర్తించడం). Topic 9.
- **"How would you debug 'website is slow / not loading'?"** — ping (reachable?), dig (DNS?), curl -I (response/status?), traceroute (where stuck?), netstat (port?). Systematic layer-by-layer.
- **"Forward vs reverse proxy?"** — Forward = client కోసం (client hide, office filtering). Reverse = server కోసం (Nginx, load balance, SSL termination, backend hide).
- **"L4 vs L7 load balancer?"** — L4 = TCP/IP+port, content చూడదు, fast. L7 = HTTP, URL/header/cookie ఆధారంగా smart routing (`/api` →A, `/static` →B), SSL termination. WebSockets/sticky sessions → IP hash. Health checks తో down servers తీసేస్తుంది.
- **"XSS vs CSRF?"** — XSS = malicious JS inject (sanitize, CSP, HttpOnly). CSRF = user తెలియకుండా request (CSRF token, SameSite cookie).
- **Gotcha:** ping fail = server down కాదు (ICMP block అవ్వొచ్చు firewall లో). HTTP reachability కి curl వాడు.
- **Gotcha:** HTTPS transit encryption మాత్రమే — server-side vulnerabilities (SQLi, XSS) ని ఆపదు. Security = multiple layers (defense in depth).

---

13. Interview Q&A + Memory Tips + Common Mistakes

వివరణ

ఇది మొత్తం guide యొక్క **revision + interview cheat sheet.** Interview ముందు రోజు ఈ ఒక్క topic చదివితే అంతా గుర్తొస్తుంది. మూడు భాగాలు: **(1) rapid-fire Q&A** (topic-wise), **(2) memory tips/mnemonics table**, **(3) common mistakes** (interview లో మాట్లాడకూడని తప్పులు). ఇది reference — bookmark చేసుకో.

Part A — Rapid-Fire Q&A (interview లో 30-second answers)

Basics & Models:

ప్రశ్న	సమాధానం (short)
Bandwidth vs Latency?	Bandwidth = capacity (Mbps, ఎంత). Latency = delay (ms, ఎంత time).
OSI layers ఎన్ని, ఏవి?	7: Application, Presentation, Session, Transport, Network, Data Link, Physical.
OSI vs TCP/IP?	OSI = 7 theoretical. TCP/IP = 4 practical (internet reality).
Encapsulation అంటే?	ప్రతి layer తన header జోడించడం (top→down). Data→Segment→Packet→Frame→Bits.
PDU పేర్లు?	Segment (L4), Packet (L3), Frame (L2), Bit (L1).
Router ఏ layer? Switch? Hub?	Router = L3 (IP), Switch = L2 (MAC), Hub = L1 (broadcast).

Addressing & Network layer:

ప్రశ్న	సమాధానం
MAC vs IP?	MAC = L2, 48-bit, permanent, local (who). IP = L3, 32/128-bit, changeable, global (where).
ARP పని?	IP → MAC translation (broadcast request, unicast reply).
IPv4 vs IPv6?	32-bit (~4.3B) vs 128-bit (~unlimited). IPv6 కి NAT అవసరం లేదు.
Subnetting usable hosts?	2^(host bits) − 2 (network + broadcast తీసేయాలి).
/26 = ఎన్ని hosts?	host bits 6 → 64−2 = 62 usable, block size 64.
NAT ఎందుకు?	Private IPs → ఒక public IP (IPv4 save + security).
DHCP process?	DORA: Discover, Offer, Request, Acknowledge.
127.0.0.1?	Loopback = localhost (నీ own machine).

Transport layer:

ప్రశ్న	సమాధానం
TCP vs UDP?	TCP = reliable, ordered, connection, slow (web). UDP = unreliable, fast, connectionless (video/DNS).
3-way handshake?	SYN → SYN-ACK → ACK (connection establish).
4-way termination?	FIN → ACK → FIN → ACK (close).
Handshake 3, close 4 ఎందుకు?	Open: SYN+ACK combine. Close: full-duplex, ప్రతి direction విడిగా.
Flow vs congestion control?	Flow = receiver protect (receiver window). Congestion = network protect (slow start).
TCP reliability ఎలా?	Sequence numbers + acks + retransmission + checksum + ordering.
Port ఉదాహరణలు?	80 HTTP, 443 HTTPS, 53 DNS, 22 SSH, 27017 MongoDB.

Web (HTTP/HTTPS/DNS):

ప్రశ్న	సమాధానం
HTTP stateless అంటే?	ప్రతి request స్వతంత్రం, server గత request గుర్తుంచుకోదు (cookies/tokens తో state).
Idempotent methods?	GET, PUT, DELETE (ఎన్నిసార్లైనా same). POST కాదు.
401 vs 403?	401 = not authenticated (login). 403 = authenticated కానీ no permission.
HTTP/1.1 vs 2 vs 3?	1.1 keep-alive (HOL blocking); 2 multiplexing (TCP HOL); 3 QUIC/UDP (no TCP HOL).
Cookie vs JWT?	Session cookie = server state (scale hard, revoke easy). JWT = client state (scale easy, revoke hard).
HTTPS 3 guarantees?	Confidentiality, Integrity, Authentication (CIA).
TLS symmetric+asymmetric ఎందుకు?	Asymmetric తో symmetric key share, తర్వాత fast symmetric data.
Certificate ఎలా నమ్ముతారు?	CA sign చేస్తుంది, browser root CAs ముందే నమ్ముతుంది (chain of trust).
DNS resolution steps?	cache → resolver → root → TLD → authoritative → IP.
A vs CNAME?	A = name→IP. CNAME = name→వేరే name (alias).
DNS TTL?	Record ని ఎంతసేపు cache చేయాలో (seconds).
URL enter చేస్తే?	Parse → DNS → TCP → TLS → HTTP → server → response → render.

Part B — Memory Tips & Mnemonics (గుర్తుంచుకునే tricks)

Concept	Mnemonic / Trick
OSI 7 layers (7→1)	**A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing
OSI 7 layers (1→7)	**P**lease **D**o **N**ot **T**hrow **S**ausage **P**izza **A**way
PDU units (4→1)	**S**egment **P**acket **F**rame **B**it — "SPFB"
TCP handshake	SYN, SYN-ACK, ACK — "Hello? Hello, వినిపిస్తోందా? అవును!" (phone call)
TCP termination	FIN, ACK, FIN, ACK — రెండు వైపులా విడిగా "bye"
DHCP	**DORA** — Discover, Offer, Request, Acknowledge
HTTPS guarantees	**CIA** — Confidentiality, Integrity, Authentication
Status codes	2=OK, 3=go elsewhere, 4=నీ తప్పు, 5=server తప్పు
401 vs 403	401 = "ఎవరో తెలియదు" (login), 403 = "తెలుసు కానీ NO" (permission)
MAC vs IP	MAC = who (Aadhaar), IP = where (current address)
Subnet hosts	2^host − 2 (network + broadcast తీసేయి). Block size = 256 − mask octet
TCP vs UDP	TCP = phone call (confirm), UDP = postcard (fire & forget)
DNS hierarchy	Root → TLD → Authoritative (కుడి నుండి ఎడమకి చదువు)
URL journey	Parse-DNS-TCP-TLS-HTTP-Render
Ports	80 HTTP, 443 HTTPS (443 = HTTP + "s"ecure), 22 SSH, 53 DNS

Numbers to memorize (interview లో fast recall):

```
MAC address       = 48 bits (6 bytes, hex)
IPv4              = 32 bits (4 octets, 0-255)
IPv6              = 128 bits
Port range        = 0 – 65535 (16-bit)
Well-known ports  = 0 – 1023
TCP header        = 20-60 bytes
UDP header        = 8 bytes
HTTP port         = 80,   HTTPS = 443
DNS port          = 53 (UDP)
/24 subnet        = 256 total, 254 usable
```

Part C — Common Mistakes (interview లో మాట్లాడకూడనివి)

❌ తప్పు	✅ సరైనది
"TCP fast, UDP slow"	UDP faster (overhead తక్కువ). TCP reliable కానీ slower.
"IP is layer 4"	IP = Layer 3 (Network). TCP/UDP = Layer 4.
"Switch and router same"	Switch = L2 (MAC, same network). Router = L3 (IP, వేర్వేరు networks).
"MAC address మారుతుంది"	MAC = permanent (hardware). IP మారుతుంది.
"HTTPS = పూర్తి security"	HTTPS = transit encryption మాత్రమే. Server-side (SQLi, XSS) ఇంకా vulnerable.
"SSL is current"	SSL deprecated. TLS 1.2/1.3 current (జనాలు అలవాటుగా "SSL" అంటారు).
"DNS uses TCP"	DNS = UDP (53) primarily. పెద్ద responses TCP.
"401 = no permission"	401 = not authenticated. 403 = no permission.
"usable hosts = 2^n"	2^n − 2 (network + broadcast reserved).
"handshake 4 steps"	Open = 3-way. Close = 4-way. తికమక పడొద్దు.
"encapsulation = encryption"	Encapsulation = headers wrapping (అన్ని packets). Encryption = TLS (ప్రత్యేకం).
"HTTP/2 = UDP"	HTTP/2 = TCP. HTTP/**3** = UDP (QUIC).
"POST is idempotent"	POST కాదు. GET/PUT/DELETE idempotent.
"ping fail = server down"	ICMP block అవ్వొచ్చు. curl తో HTTP check.
"cookie stores password"	Cookie = session ID/token (password కాదు). Server లో hash.

Part D — Full-Journey Answer (interview లో చెప్పే model answer)

"URL enter చేస్తే ఏం జరుగుతుంది?" — ఈ answer ని కంఠస్థం చెయ్యి:

> "మొదట browser **URL ని parse** చేస్తుంది — scheme (https), host (github.com), port (443), path. తర్వాత **DNS resolution** — browser/OS cache చూసి, లేకపోతే recursive resolver → root → TLD → authoritative server ద్వారా host name ని IP గా మారుస్తుంది. IP దొరికాక, ఆ server తో **TCP 3-way handshake** (SYN, SYN-ACK, ACK) చేసి reliable connection establish చేస్తుంది, port 443 కి. HTTPS కాబట్టి **TLS handshake** — certificate verify చేసి, asymmetric encryption తో ఒక symmetric session key exchange చేసుకుంటారు. ఇప్పుడు encrypted channel మీద **HTTP GET request** పంపుతుంది. Server side లో load balancer → app server → database → HTML తయారు చేసి, **HTTP 200 response** (HTML) తిరిగి పంపుతుంది. Browser ఆ HTML ని **parse చేసి DOM, CSS ని CSSOM, render tree, layout, paint** చేసి page చూపిస్తుంది. HTML లో CSS/JS/images ఉంటే వాటికి మళ్ళీ ఇదే process (HTTP/2 multiplexing తో parallel). నా React app అయితే JS execute అయ్యి, `fetch()` calls మళ్ళీ ఈ cycle repeat చేస్తాయి."

ఈ ఒక్క answer లో నువ్వు DNS, TCP, TLS, HTTP, routing, rendering — అన్నిటినీ touch చేశావు. Interviewer ఏ step లోనైనా deep dive అడిగితే, ఆ topic కి వెళ్ళు.

Part E — Layer-wise Master Summary (ఒక్క table లో మొత్తం CN)

Layer	పేరు	Address	Device	Protocol	PDU	నీ MERN లో
7	Application	—	—	HTTP, DNS, WS	Data	fetch(), REST, Socket.IO
4	Transport	Port	—	TCP, UDP	Segment	port 443, reliability
3	Network	IP	Router	IP, ICMP	Packet	routing, NAT
2	Data Link	MAC	Switch	Ethernet, ARP	Frame	WiFi router
1	Physical	—	Hub/cable	—	Bit	WiFi signals

Key Points (మొత్తం guide సారాంశం)

- **CN = "రెండు computers ఎలా మాట్లాడతాయి".** Layered architecture (OSI 7 / TCP-IP 4) దీన్ని manageable చేస్తుంది.
- **ప్రతి layer ఒక పని:** Application (HTTP — ఏం), Transport (TCP/UDP — reliable/fast + ports), Network (IP — routing), Data Link (MAC — local), Physical (bits).
- **నీ ప్రతి `fetch()` :** DNS (name→IP) → TCP (handshake) → TLS (encrypt) → HTTP (request/response) → render. ఇదే మొత్తం CN.
- **Interview core:** OSI layers, TCP handshake, TCP vs UDP, subnetting, HTTP status codes, HTTPS/TLS, DNS resolution, "URL journey". ఈ 8 గట్టిగా ఉంటే 90% cover.
- **MERN advantage:** నువ్వు HTTP, cookies, WebSockets, fetch ఇప్పటికే వాడతావు — ఈ theory ఆ practical knowledge కి foundation ఇస్తుంది. Interview లో ఈ connection చూపిస్తే బలంగా ఉంటుంది.

Interview దృష్టి (చివరి సలహా)

- **Depth + breadth:** ఒక్కో topic ని surface గా కాకుండా "why, when, trade-off" తో చెప్పు (ఇదే SSE-level).
- **Diagrams గీయి:** TCP handshake, OSI layers, encapsulation, URL journey — whiteboard మీద గీయగలగాలి. Practice చెయ్యి.
- **MERN కి connect చెయ్యి:** "నా app లో ఇది ఇలా వాడతాను" అని చెప్తే, theory + practical రెండూ ఉన్నాయని బలంగా చూపిస్తుంది.
- **తెలియకపోతే:** "ఖచ్చితంగా తెలియదు కానీ నా అంచనా ఇది..." అని reason చెయ్యి — bluff చేయకు. Seniors reasoning చూస్తారు.
- **Revise:** interview ముందు ఈ Topic 13 ఒక్కటి చదివితే మొత్తం guide గుర్తొస్తుంది.

> **ముగింపు.** నువ్వు ఇప్పుడు — networking theory ZERO నుండి — ఒక `fetch()` వెనుక దాగిన DNS, TCP, TLS, IP routing, HTTP, encapsulation అంతా చెప్పగలవు. ఈ guide లోని ప్రతి analogy (postal system, phone call, రోడ్లు, passport), ప్రతి diagram, ప్రతి table — "ఒకసారి చదివితే జీవితంలో మర్చిపోకూడదు" అనే లక్ష్యంతో రాసినవి. Interview ముందు Topic 13 revise చెయ్యి, whiteboard మీద diagrams practice చెయ్యి. నీ MERN అనుభవం + ఈ CN foundation = SSE-ready. All the best! 🚀
>
> **CS-fundamentals set:** `OOPS_Telugu.md` , `GO_Telugu.md` , `HLD_Go_Telugu.md` , `LLD_Go_Telugu.md` , `SystemDesign_Go_Telugu.md` , `DSA_00..10` , `JavaScript_Telugu.md` , మరియు ఈ `CN_Telugu.md` . అన్నీ కలిపి — complete interview preparation.